

Online Post-Quantum Voting Application: Verifiable with Board Privacy, and Coercion Resistant

Anonymous Author(s)

Abstract

Electronic voting need two properties at once, which are secrecy of each vote and public evidence that the announced outcome matches the ballots that were cast. Currently, all of the deployed voting systems obtains them from discrete logarithms or from factoring, so a quantum adversary that stores the bulletin board today can open the ballots of an old election later. The post-quantum replacements designed so far fall short in three ways. Their zero-knowledge proofs are analyzed only in the classical random oracle model, and the works behind the most efficient lattice mix-net state that new techniques are needed against a quantum adversary. None of them resists a coercer who forces a voter to reveal voting material. Their tally is dominated by an exact shortness proof and by a ballot encrypted twice so that the proof can extract it.

In this work we present Oylama, a voting system built only from standardized primitives, which are threshold ML-KEM, ML-DSA, SHAKE256, AES-256-GCM and a lattice mix-net, and we prove that it satisfies ballot privacy against an adversarial board and strong verifiability in the quantum random oracle model. We then present Oylama, the first post-quantum coercion resistant scheme, which keeps the cleansing-hiding structure of CHide (IEEE CSF'24) and evaluates its circuit on packed plaintexts so that one distributed decryption serves N parallel binary gates. Four changes to the mixing path, a module in place of one large ring, a three element ballot witness, a folded mixing commitment and batched decryption proofs, lower the tally time by a factor 5.6 and the transcript by a factor 6.0 against the other mix-net schemes. Oylama is the first online post-quantum voting system; in fact, none of the existing post-quantum voting schemes has been ever implemented as practical application.

1 Introduction

Elections carried out over a network must keep each vote secret while letting anyone confirm that the announced outcome matches the ballots that were cast [1–3]. The cryptographic layer that provides both properties in every deployed system is built from discrete logarithms or from factoring [1, 2, 4], so an adversary that stores today's bulletin board and waits for a quantum computer can open the ballots of an election held decades earlier [5]. Post-quantum replacements have therefore been studied for a decade [6–9], and lattice assumptions are the usual starting point [10–12]. Two questions have been left open by that line of work, and a third concerns its cost.

Problem 1: security of the proofs against quantum adversaries. Each lattice voting system in the literature makes its zero-knowledge proofs non-interactive with the Fiat–Shamir transform and analyzes them in the classical random oracle model, which is not sound once the adversary can query the hash function in superposition [13]. Del Pino et al. [7] note that a reduction in the quantum random oracle model would need a different instantiation and would raise the size of the proofs and of the public keys noticeably, and Aranha

et al. [14] state that new techniques would have to be developed before the bounded-opening proof of their mix-net could be proved secure against quantum adversaries, that quantum security for all of their proofs would ask for a commit-and-open replacement of the homomorphism-based ones, and that such a replacement would cost significantly in proof size and prover time. The problem is to obtain a lattice voting system whose proofs are provably secure in that model without paying the price. Since privacy against an honest ballot box only is of limited use in the remote setting [15, 16], we ask the guarantees to be privacy against a board chosen by the adversary and strong verifiability [3].

Problem 2: post-quantum coercion resistance. A voter forced to hand over voting material should still be able to cast the ballot of their choice. The scheme of Juels et al. [17] achieves this with fake credentials and has been refined many times [18, 19], most recently by Cortier et al. [20], who showed that the cleansing phase of that family leaks more than the number of removed ballots and gave the cleansing-hiding variant CHide with a definition that accounts for revoting and for ballots injected by authorities. Every such construction uses re-randomizable encryption over a group of prime order together with plaintext equality tests, so the whole family fails against a quantum adversary. Boyen et al. [8] state that instantiating these techniques with practical lattice primitives is an open question that they leave for future work, and, as we identified in Section 2, none of the thirteen post-quantum voting schemes published so far satisfies any coercion resistance definition.

Problem 3: the cost of the tally. Lattice mix-nets are the only route to a post-quantum tally that supports arbitrary ballot structures, and the construction of [14] is the most efficient one available. Its mixing transcript is dominated by an exact amortized proof that the re-randomization randomness is short, and its ballots have to be encrypted under two keys so that the security proof can extract them. Both costs should be removed without weakening the guarantees.

Contributions. We make the following contributions.

- We address Problem 1 with Oylama, a voting scheme built only from standardized building blocks of NIST, which are threshold ML-KEM [21–23], ML-DSA [24], SHAKE256 [25], AES-256-GCM [26, 27] and the lattice mix-net of [14]. We replace the two proof families that resist a quantum analysis by ones that admit a lossy instance and an online extractor, and we prove that Oylama satisfies delayed universal verification ballot privacy against a malicious board and strong verifiability in the quantum random oracle model (Section 5).
- We address Problem 2 with Oylama, the first post-quantum coercion resistant voting scheme. It keeps the cleansing-hiding structure of [20] and replaces the interactive gate of that scheme by a packed lattice evaluation in which a single distributed decryption serves N parallel binary gates, where N is the ring degree (Section 8).

- We address Problem 3 with four changes to the mixing path, which are a module of rank k in place of one ring of degree kN , a ballot proof whose witness has three elements instead of four in place of the double encryption, a mixing commitment folded onto a single message slot, and batched linear proofs for the decryption shares. We compare Oylama against thirteen post-quantum voting schemes, first by computing the time and the space complexity of each procedure from scratch and then by implementing the cryptographic core of all of them in one framework (Sections 6 and 7).

For the full proofs of theorems and derivation of the parameter set, refer to appendix of full version at <https://github.com/ethanwaker9/work-36/blob/main/full/full.pdf>.

Threat model. The trustees hold a (t, n_T) sharing of the election key and at most $t - 1$ deviate, at least one of the n_M mix servers is honest, the registrar and the board are not dishonest at the same time, which is the assumption under which strong verifiability is stated [3], and the voters of a designated subset run their own check. For Oylama we also assume, as in [17, 20], an untappable registration channel and an anonymous submission channel.

2 Related Work

Two tallying methods recur [28, 29]: *homomorphic* tallying aggregates encrypted votes into a sum decrypted once, whereas a *mix-net* anonymizes ciphertexts through a shuffle before decrypting each ballot.

Quantum-resistant voting systems have diverse structures but minimal privacy and verifiability guarantees. Table 1 summarizes their building blocks and guarantees, and here we record the definitional and cryptanalytic gaps that place every one of them below CR.

Epoque [8] proves privacy under *Vote Privacy* [30], which Bernhard et al. [31] show to be weak, since it restricts the class of protocols it applies to, can be inconsistent under natural result functions such as majority [32], fails for protocols emitting auxiliary data such as ElectionGuard’s tally proofs [4], and misses violations beyond same-vote detection. Lemma 5 of [29] also omits the hybrid argument for secret-share uniformity under commitment hiding, its verifiability definition [33] does not distinguish ballot deletion from modification [34], and concrete post-quantum parameters are absent. The *NTRU mix-net* [9] must avoid the overstretched regime $q > 0.0058 \cdot \sigma^2 \cdot d^{2.484}$ that enables subfield lattice attacks, rests on one honest mix server, and is not a complete scheme, since its privacy is referred to Aranha et al. [35], whose own theoretical system [28, App. H] provides no proofs. *FOO-based voting* [36], combining threshold lattice blind signatures [37–39] with Dual-Regev encryption [40, 41], rests on a conjectural one-more unforgeability assumption [42] and breaks if its anonymous channel is compromised, while *code-based voting* [43] relies on the less-studied dual-LPN assumption, has uncertain 128-bit parameters against ISD attacks [44], and lacks formal privacy proofs.

EVOLVE [7] lacks individual verifiability, needs a large voter threshold, and supports no revoting, and its modification of BPRIV [31] sidesteps the simulator requirement by making the tally consistent with the board in both worlds, whereas standard BPRIV with $\beta = 1$ requires a simulator producing a fake proof consistent with BB_1 that

EVOLVE never constructs, while it also proves only a weaker “realistic result” consistency. *EVOLVED* [45] adds Hamming-weight proofs but inherits these gaps. *LWE-Voting* [46] leaves individual verifiability open and obtains universal verifiability only if its two Lindner–Peikert [47] verification servers do not collude, an assumption never formalized in the KTV trust model, and which if violated also falsifies its privacy argument (Theorem 1 of [46]), while its $n = 512$ parameter has degraded under updates to the BKZ estimator [48]. Among the remainder, Chillotti et al. [6] give only a proof sketch for ballot privacy; Abapour et al. [49] do prove it, via a lattice polynomial commitment [50] and a posteriori-secure [51] non-interactive blind signature [52], but their economic structure lacks flexibility for general use [53]; Kyber-BC [54] lets a central bank hold every per-voter KEM [55] secret key while ciphertexts sit on-chain [56–58]; Hybrid-BC [59] gates ECDH-encrypted [60] votes behind a smart-contract ABE policy [61, 62]; and Gabriel et al. [63] embed multivariate-encrypted [64, 65] ballots into cover images by LSB steganography. None of the twelve provides any guarantee above BPRIV.

Finally, Cardoso Barbosa et al. [66] port Helios to lattices with the shuffle of known values of [67] over BDLOP commitments [68] and a sketched proof for the sum of commitments, with concrete parameters and a public repository; that work is an implementation study and states neither a privacy nor a verifiability theorem.

3 Preliminaries

Notation. For a power of two N let $R = \mathbb{Z}[X]/(X^N + 1)$ and $R_q = R/qR$, and let $\|a\|_\infty$ and $\|a\|_2$ be the largest absolute value and the Euclidean norm of the coefficient vector of a in the centered range $(-q/2, q/2]$. Throughout, λ is the computational and sec the statistical security parameter, n is the number of ballots, n_C the number of options, n_T and t the number of trustees and the quorum, n_M the number of mix servers, and $\text{negl}(\lambda)$ a negligible function.

Assumptions. $\text{MLWE}_{N,q,k,\chi}$ asks to distinguish $(a, \langle a, s \rangle + e)$ from uniform for $a \leftarrow R_q^k$ and $s, e \leftarrow \chi$, and $\text{MSIS}_{N,q,k,\beta}$ asks for a nonzero z with $\|z\|_\infty \leq \beta$ and $Az = 0$. Both reduce from worst-case module lattice problems [10, 12], χ is the ternary distribution as in [14], and concrete hardness is estimated with the lattice estimator [48] and the homomorphic encryption parameter tables [86].

Encryption, commitments and proofs. We use BGV [87] over a module of rank k : the public key is (A, t) with $A \leftarrow R_q^{k \times k}$ expanded from a SHAKE256 seed and $t = As + pe$ for ternary s, e and plaintext modulus p ; an encryption of $\mu \in R_p$ is $(u, w) = (A^T r + pe_1, t^T r + f_0)$ with ternary r, e_1, e_0 and $f_0 = \mu + pe_0$, and decryption returns $[w - s^T u]_q \bmod p$. The instantiation of [14] is the case $k = 1$ over a ring of degree kN . Ciphertexts add, so an encryption of zero re-randomizes. Commitments are the module scheme of [68], which is hiding under MLWE, binding under MSIS and linearly homomorphic. All proofs are Fiat–Shamir transformations of sigma protocols with aborts [68, 88]: Π_{Lin} shows that two commitments open to values related by $x' = \alpha x + \beta$ for public α, β , Π_{Bnd} is its amortized version for many short preimages of one matrix, and Π_{Shuf} [67] shows that a list of commitments opens to a permutation of a public list using one instance of Π_{Lin} per entry. The mix-net Π_{Mix} and the verifiable distributed decryption Π_{Dec} of [14] are built from these.

Scheme	Building Blocks	Privacy	Verifiability	Revoting	Parameters	Code
Epoque [8]	IBE [69], Com [70], NIZK [71]	Weak BPRIV [30]	indiv, univ	×	✓	p
NTRU-Mix [9]	PKE [72], Com [73], ZKP [74–76]	×	univ	×	✓	✓
FOO-based [36]	TPKE [40, 41], BSign [37–39]	×	partial univ	×	×	×
Code-based [43]	PKF[77, 78], Com[43], NIZK[43, 79]	×	indiv, univ	×	✓	×
EVOLVE [7]	PKF[7], ZKP[80], Sig[81], Com[68]	BPRIV [31]	partial univ	×	✓	p
EVOLVED [45]	PKF[45], Com[7], HW-Proof[82]	×	partial univ	×	✓	×
Homo-LWE [6]	FHE [83], Sign [84], NIZK [85]	BPRIV [31]	indiv, univ	✓	×	×
BPRIV-BC [49]	Com[50], PostSec[51], NIBSign[52]	BPRIV [31]	indiv, univ	×	×	×
Kyber-BC [54]	KEM[55], Block[56, 57], Hash[58]	×	partial indiv	×	✓	p
Hybrid-BC [59]	ABE[59], ECDH[60], Block[61, 62]	×	partial indiv	×	×	p
Steg-MQPC [63]	MQPC [64, 65], LSB-Steg [63]	×	×	×	✓	p
LWE-Voting [46]	PKF[47], ZKP[46], Sign [46]	×	univ	×	✓	p
Helios-PQ [66]	Com [68], Shuf [67]	×	×	×	✓	✓
Oylama (Ours)	TML-KEM[21, 22], ML-DSA[24], AES-256-GCM[26], Mix[14]	du-mb-BPRIV [16]	strong [3]	✓	✓	✓
Oylama (Ours)	Oylama + OblSort, PackMPC	CR [20]	strong [3]	✓	✓	✓

Table 1: Comparison of post-quantum electronic voting schemes, with the two schemes of this work in the last rows. Parameters column: ✓ if the paper provides a concrete numerical parameter set (security level, modulus, dimensions), × if only asymptotic or abstract parameter formulas are given. Code column: ✓ if a publicly accessible implementation URL is given, p if the authors report having implemented the scheme but publish no source, × if no implementation exists. Building-block abbreviations are the standard ones (IBE, Com, PKE, TPKE, KEM, ABE, ECDH, Sig, BSign, NIBSign, ZKP/NIZK, Hash, Block), together with deniable FHE dFHE, polynomial commitment PCOM, posterior security PostSec, Hamming-weight proof HW-Proof, multivariate quadratic polynomial cryptography MQPC, and least-significant-bit steganography LSB-Steg.

Standardized primitives. ML-KEM-768 [21], ML-DSA-65 [24], the SHAKE256 [25] and AES-256-GCM [26, 27] provide key encapsulation, signatures, each hash and challenge with domain separation, and each data encapsulation. Threshold decapsulation follows [22, 23], which share the secret with a replicated scheme whose reconstruction coefficients are 0 and 1, so no Lagrange denominator has to be inverted, and hide an honest share inside a flooding term; we reuse that sharing for the BGV key. In the quantum random oracle model [13] we use two properties of this family: a transcript is simulated by adaptively reprogramming the oracle at an unpredictable point [89, 90], and the witness is extracted by the measure-and-reprogram technique [91] with a quadratic loss in the number of oracle queries.

4 Security Definitions

4.1 Ballot Privacy against a Malicious Board

The most fundamental voting privacy property that has been well-studied is BPRIV[31] (no entity can learn the content of ballots), and a stronger notion is mb-BPRIV [15] (ballot privacy against an adversary who controls the ballot box). du-mb-BPRIV is an equivalent of mb-BPRIV, where verification is delayed until after tallying[16], which we focus on here.

Definition 4.1 (du-mb-BPRIV [16]). Let $\mathcal{V}$ be a voting system, Recover a recovery algorithm, and $\mathcal{I} = \mathcal{H} \cup \mathcal{D}$ a partition into honest and dishonest voters with $\mathcal{H}_{\text{check}} \subseteq \mathcal{H}$ the verifying subset. A *selection function* for integers $m, n \geq 1$ is a mapping $\pi : [n] \rightarrow [m] \cup (\{0, 1\}^* \times \{0, 1\}^*)$, inducing a board transformation $\tilde{\pi}$; the recovery algorithm Recover(BB, BB_1) returns such a π capturing how $\mathcal{A}$ constructed BB from BB_1 . We say $\mathcal{V}$ satisfies du-mb-BPRIV w.r.t. Recover if there exists an efficient simulator Sim such that for every efficient adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{du-mb-bprv}}(\lambda) = \left| \Pr[\text{Exp}^0 = 1] - \Pr[\text{Exp}^1 = 1] \right| = \text{negl}(\lambda)$, where $\text{Exp}^\beta := \text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, \text{Recover}}^{\text{du-mb-BPRIV}, \beta}(\lambda)$ is defined in Figure 1.

In the experiment, $\mathcal{O}_{\text{voteLR}}$ computes ballots for both v_0, v_1 and stores them in BB_0, BB_1 , while the voter state keeps st_{pre}^β , which depends on the secret bit, and st_{post}^0 , which always comes from $\beta = 0$ because recovery makes the tally reflect the left board. After $\mathcal{A}$ outputs BB it guesses d^* before seeing the tally; the single call to $\mathcal{O}_{\text{tally}}$ tallies BB when $\beta = 0$ and tallies Recover(BB, BB_0, BB_1) with a simulated proof when $\beta = 1$; and if an expected verification is missing or fails the experiment returns d^* or a random bit, so tampering that causes a detectable complaint gives no advantage.

4.2 Verifiability

Generally, it comes from the fact that voters can verify their ballots were recorded and anyone can verify correct tallying, but we consider the *Strong Verifiability* notion by [3] here, which covers broader spectrum.

Definition 4.2 (Strong Verifiability). Let $\mathcal{V} = (\text{Setup}, \text{Credential}, \text{Vote}, \text{Validate}, \text{Box}, \text{VerifyVote}, \text{Tally}, \text{Verify})$ be a voting protocol with admissible vote space $\mathbb{V}$, result space $\mathcal{R}$, and result function $\rho : \mathbb{V}^r \rightarrow \mathcal{R}$ admitting partial tallying via a commutative operation $\oplus_R$, i.e. $\rho(S_1 \cup S_2) = \rho(S_1) \oplus_R \rho(S_2)$ for disjoint vote lists $S_1, S_2 \subseteq \mathbb{V}^*$. Both experiments maintain: a public credential set L ; a table U of registered tuples (id, upk, usk) ; a set CU of corrupted voter pairs (id, upk) ; a list $HVote$ of triples (id, v, b) from honest voting queries (last ballot per id only); and $Checked \subseteq HVote$ where $CheckedVoter(id, v, b) = 1$. The oracles are: $\mathcal{O}_{\text{reg}}(id)$: honestly generates $(upk, usk) \leftarrow \text{Credential}(1^\lambda, id)$, returns upk , updates L, U . $\mathcal{O}_{\text{corrupt}}(id)$: reveals (upk, usk) for registered id , updates the CU . $\mathcal{O}_{\text{vote}}(id, v)$: for honest, uncorrupted id with $v \in \mathbb{V}$, returns $b \leftarrow \text{Vote}(id, upk, usk, v)$, updates $HVote$. $\mathcal{O}_{\text{cast}}(id, b)$: runs $\text{Box}(BB, b)$ on an honest bulletin board.

The experiments are given in Figure 2. In both, the adversary wins if it produces a verified tally that cannot be expressed as $\rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$, where $\{v_i^E\}$ are votes from $Checked$, $\{v_i^A\}$ are

Exp _{$\mathcal{A}, \mathcal{V}, \text{Sim_Recover}$} ^{du-mb-BPRIV, β} (λ)

```

1:  $Checked, Happy \leftarrow \emptyset; V, PU, U, CU \leftarrow \text{empty}$ 
2:  $(pd, sd) \leftarrow \text{Setup}(1^\lambda)$ 
3: for  $id \in \mathcal{I}$  do
4:    $(pc, sc) \leftarrow \text{Register}(id); U[id] \leftarrow sc; PU[id] \leftarrow pc$ 
5:   if  $id \in \mathcal{D}$  then  $CU[id] \leftarrow U[id]$ 
6:  $BB \leftarrow \mathcal{O}_{\text{voter}, R, \mathcal{O}_{\text{board}}}(pd, CU, PU, \mathcal{H}_{\text{check}})$ 
7: if  $\mathcal{H}_{\text{check}} \not\subseteq V \vee \text{ValidBoard}(BB, pd) = \perp$  then return  $d \leftarrow \{0, 1\}$ 
8:  $d^* \leftarrow \mathcal{A}()$ 
9:  $(r^*, \Pi^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{tally}}}()$ 
10: if  $\text{VerifyTally}((pd, pbb, r^*), \Pi^*) = \perp$  then return  $d \leftarrow \{0, 1\}$ 
11:  $d \leftarrow \mathcal{A}^{\mathcal{O}_{\text{verify}}}()$ 
12: if  $\mathcal{H}_{\text{check}} \not\subseteq Checked$  then return  $d \leftarrow \{0, 1\}$ 
13: if  $\mathcal{H}_{\text{check}} \not\subseteq Happy$  then return  $d^*$ 
14: return  $d$ 

```

$\mathcal{O}_{\text{board}}()$

```

1: return  $\text{Publish}(BB_p)$ 

```

$\mathcal{O}_{\text{voter}, R}(id, v_0, v_1)$

```

1: if  $id \notin \mathcal{H}$  then return  $\perp$ 
2: for  $i \in \{0, 1\}$  do  $(pc, b_i, st_{\text{pre}}^i, st_{\text{post}}^i) \leftarrow \text{Vote}(pd, pc, v_i)$ 
3:  $V[id] \leftarrow V[id] \parallel (st_{\text{pre}}^\beta, st_{\text{post}}^0, v_0)$ 
4:  $BB_0 \leftarrow BB_0 \parallel (id, (pc, b_0)); BB_1 \leftarrow BB_1 \parallel (id, (pc, b_1))$ 

```

$\mathcal{O}_{\text{tally}}()$

```

1: if  $\beta = 0$  then  $(r, \Pi) \leftarrow \text{Tally}(BB, pd, sd); \text{return}(r, \Pi)$ 
2:  $BB' \leftarrow \text{Recover}(BB, BB_0, BB_1)$ 
3:  $(r, \Pi) \leftarrow \text{Tally}(BB', pd, sd)$ 
4: return  $(r, \text{Sim}(pd, \text{Publish}(BB), r))$ 

```

$\mathcal{O}_{\text{verify}}(id)$ for $id \in \mathcal{H}_{\text{check}}$

```

1:  $Checked \leftarrow Checked \cup \{id\}$ 
2: if  $\text{VerifyVote}(id, st_{\text{pre}}, st_{\text{post}}, BB, pc, sc) = \top$  then
3:    $Happy \leftarrow Happy \cup \{id\}$ 
4: return  $Happy$ 

```

Figure 1: The du-mb-BPRIV security experiment. $\mathcal{I} = \mathcal{H} \cup \mathcal{D}$ with $\mathcal{H}_{\text{check}} \subseteq \mathcal{H}$.

Exp _{$\mathcal{A}, \mathcal{V}$} ^{ver-b}(λ)

```

1:  $I, U, CU, HVote \leftarrow \emptyset$ 
2:  $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$ 
3:  $(BB, \Pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{reg}}, \mathcal{O}_{\text{corrupt}}, \mathcal{O}_{\text{cast}}}(pk)$ 
4: if  $\text{Verify}(BB, \Pi) = 0 \vee p = \perp$  then return 0
5: if  $\exists \{(id_i^A, v_i^A, \star)\} \subseteq HVote, \exists \{v_i^B\} \subseteq V, |\{v_i^B\}| \leq |CU|$ :
6:    $p = \rho(\{v_i^A\}) \oplus_{RP}(\{v_i^B\}) \oplus_{RP}(\{v_i^A\})$  then return 0
7: return 1

```

Exp _{$\mathcal{A}, \mathcal{V}$} ^{ver-r}(λ)

```

1:  $I, U, CU, HVote \leftarrow \emptyset$ 
2:  $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$ 
3:  $(p, \Pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{reg}}, \mathcal{O}_{\text{corrupt}}, \mathcal{O}_{\text{cast}}}(pk)$ 
4: if  $\text{Verify}(BB, \Pi) = 0 \vee p = \perp$  then return 0
5: if  $\exists \{(id_i^A, v_i^A, \star)\} \subseteq HVote, \exists \{v_i^B\} \subseteq V, |\{v_i^B\}| \leq |CU|$ :
6:    $p = \rho(\{v_i^A\}) \oplus_{RP}(\{v_i^B\}) \oplus_{RP}(\{v_i^A\})$  then return 0
7: return 1

```

Figure 2: Exp^{ver-b} (left): $\mathcal{A}$ controls trustee, voters, and bulletin board; registrar is honest. Exp^{ver-r} (right): $\mathcal{A}$ controls trustee, voters, and registrar; bulletin board is honest ($\mathcal{A}$ adds ballots only via $\mathcal{O}_{\text{cast}}$, does not output BB).

from some subset of $HVote \setminus Checked$, and $\{v_i^B\} \subseteq V$ with $|\{v_i^B\}| \leq |CU|$. We say $\mathcal{V}$ achieves *strong verifiability* if for all PPT $\mathcal{A}$, both $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{ver-b}}(\lambda) = 1]$ and $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{ver-r}}(\lambda) = 1]$ are negligible, i.e. under the assumption that the registration authority and bulletin board are not simultaneously dishonest.

4.3 Coercion Resistance

The strongest coercion resistance notion is the one introduced by Cortier et al. alongside CHide [20], which we restate because Oylama is proved secure against it.

A voting scheme in the CHide framework is a tuple $\Pi = (\text{Setup}, \text{Register}, \text{Vote}, \text{Valid}, \text{FakeCred}, \text{Tally}, \text{Verify})$ together with a behavioral distribution $\mathcal{B}$ over sequences of pairs (i, v) . Indices i in the natural voter range $[1, n_V]$ denote ballots cast by registered voters (possibly multiple times, modeling revoting), and fresh *negative* indices denote ballots posted with fabricated credentials by authorities, registrars, or adversaries. The absence of any pair $(i, \star)$ for a given i models abstention. The function Cleanse resolves multiple ballots per voter according to the protocol’s re-voting policy, and the deterministic Result function then maps the cleansed multiset to an election outcome. The scheme has no algorithm for opening an already posted ballot to a different value because under coercion the voter is allowed only to substitute a fresh fake credential through FakeCred. The definition takes the form of two experiments, $\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Real}}$ and $\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Ideal}}$, which the full version states line by line.

Definition 4.3 (Coercion Resistance [20]). A voting scheme Π satisfies *Coercion Resistance* if for every PPT adversary $\mathcal{A}$, every choice of parameters n_T, t, n_V, n_A, n_C , and every behavioral distribution $\mathcal{B}$,

there exists a PPT adversary $\mathcal{B}^*$ and a negligible function $\mu(\cdot)$, for the two experiments described below, such that

$$\left| \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Real}}(\lambda, n_T, t, n_V, n_A, n_C, \mathcal{B}) = 1] - \Pr[\text{Exp}_{\Pi, \mathcal{B}^*}^{\text{CR-Ideal}}(\lambda, n_V, n_A, n_C, \mathcal{B}) = 1] \right| \leq \mu(\lambda).$$

The Real experiment runs the protocol honestly. The challenger samples a bit β , builds the multiset B from $\mathcal{B}$, and applies an obey-or-evade adjustment: under $\beta = 1$ the coerced voter is removed from B entirely (the coercer is expected to vote with the disclosed credential); under $\beta = 0$ all of voter j ’s entries are removed except the last, which is replaced by (j, α) if $\alpha \neq \phi$. The disclosed credential cred is exposed to the adversary *at the moment casting closes*: it equals the genuine cred_j on $\beta = 1$ and a freshly sampled $\text{FakeCred}(\text{cred}_j)$ on $\beta = 0$. The adversary then casts any final batch of ballots before the tally is computed. The Ideal experiment skips the protocol entirely: only $|D|$, the corrupted-voter assignments, and the published r^* are leaked, and no credential is ever disclosed because the model assumes the coercer departs once casting closes.

5 The Oylama Voting System

5.1 Entities and Trust Assumptions

Oylama has five types of roles: the *registrar* $\mathcal{R}$ issues one signing credential per voter, the *trustees* $T_1, \dots, T_{n_T}$ hold the decryption key under a (t, n_T) replicated sharing, the *bulletin board* $\mathcal{B}$ stores ballots and returns its public view, the *mix servers* $S_1, \dots, S_{n_M}$ shuffle in sequence, and the *auditors* recompute the public checks. Privacy holds when at most $t - 1$ trustees deviate, at least one mix server is honest and the board is arbitrary, which is what separates Oylama from earlier post-quantum schemes, and verifiability holds when the registrar and the board are not dishonest at the same time, as in Definition 4.2. The registrar delivers each signing key through an ML-KEM encapsulation that keys an AES-256-GCM channel, and SHAKE256 produces every hash, challenge and public random string.

5.2 Ballot Structure and the Witness Elements

A vote for option $v \in [0, n_C)$ is encoded as the plaintext $\mu_v \in R_p$ whose first 16 coefficients carry the binary expansion of v , whose next 224

carry SHAKE256(v) truncated to 224 bits and whose remaining coefficients are zero, so decoding recomputes the tag and the padding and discards a ciphertext that decrypts to an arbitrary element of R_p .

Oylama encrypts over a module of rank k above the ring $R = \mathbb{Z}[X]/(X^N + 1)$ with $N = 256$ rather than over one ring of degree kN , which is the shape of ML-KEM and the first of four levers. The public key is $pk = (A, t)$ with $A \in R_q^{k \times k}$ expanded from a SHAKE256 seed and $t = As + pe$, and a ciphertext is $c = (u, w) \in R_q^k \times R_q$. At a fixed lattice dimension $D = kN$ a ring ciphertext holds $2D$ coefficients and a module one holds $D + N$, so the ciphertext shrinks by $2k/(k+1) = 1.88$ and the opening below by $3k/(2k+1) = 1.45$ at $k = 16$, while the lattice problems are unchanged because they depend on D and q alone.

The ballot is $b = (c, \pi, \sigma)$ with $c = \text{BGV.Enc}_{pk}(\mu_v)$, σ the ML-DSA signature over the election identifier, the identity and (c, π) , and π a proof of knowledge of an opening of c . Earlier lattice mix-net voting either encrypts under two keys and proves plaintext equality or proves knowledge of the four elements (r, e_0, e_1, μ) [14]; Lemma 5.1 shows that three are enough.

LEMMA 5.1. *Let $pk = (A, t)$ with $t = As + pe$ and let $c = (u, w) \in R_q^k \times R_q$. If there are $r, e_1 \in R^k$ and $f_0 \in R$ with $u = A^T r + pe_1$ and $w = t^T r + f_0$ in R_q , then $w - s^T u = f_0 + p(e^T r - s^T e_1)$ in R_q , the decryption of c under s equals $f_0 \bmod p$, and the decryption noise obeys $\|w - s^T u\|_\infty \leq \|f_0\|_\infty + p \sum_{i \leq k} (\|e_i\|_2 \|r_i\|_2 + \|s_i\|_2 \|e_{1,i}\|_2)$.*

PROOF. Substituting the hypotheses gives $w - s^T u = t^T r + f_0 - s^T (A^T r + pe_1) = (t - As)^T r + f_0 - ps^T e_1$, and $t - As = pe$ in R_q by the definition of the public key, so this equals $f_0 + p(e^T r - s^T e_1)$; reducing modulo p removes the second summand, so the decryption returns $f_0 \bmod p$. For the norm, the triangle inequality gives $\|w - s^T u\|_\infty \leq \|f_0\|_\infty + p \sum_i \|e_i r_i\|_\infty + p \sum_i \|s_i e_{1,i}\|_\infty$, and for any $x, y \in R$ the coefficient of X^i in xy is an inner product of a permutation of the coefficients of x with a signed permutation of those of y , so $\|xy\|_\infty \leq \|x\|_2 \|y\|_2$ by Cauchy–Schwarz; applying this to each $e_i r_i$ and each $s_i e_{1,i}$ gives the claim. $\square$

So an opening (r, e_1, f_0) fixes both the plaintext and the noise, and the message and the second noise term never enter the witness. The ballot proof Π_{Ball} is a Fiat–Shamir proof with aborts for the relation

$$\left\{ ((u, w), (r, e_1, f_0)) : u = A^T r + pe_1, w = t^T r + f_0, \|(r, e_1, f_0)\|_\infty \leq B_{\text{bal}} \right\},$$

whose response has $2k + 1$ ring elements instead of $3k$, which removes a third of the proof and of the prover work and removes the second ciphertext of the double encryption approach.

5.3 Mixing without an Exact Shortness Proof

In the mix-net of [14] a server commits to the randomness r'_i of the i -th re-randomization, publishes the shuffled openings, and proves with the exact amortized protocol Π_{Small} that $\|r'_i\|_\infty \leq 1$. That proof is the most expensive component of the mixing phase and the one for which no quantum analysis is available [14]. Oylama replaces it by the amortized approximate proof Π_{Bnd} on batches of τ_0 ciphertexts, which produces one response per batch and an extracted bound $12\sigma_{\text{mix}}$ with $\sigma_{\text{mix}} = 11\sqrt{k\text{cols}N\tau_0}$ instead of 1, where $\text{cols} = k_c + 2k + 1$ is the width of the mixing relation. Lemma 5.2 shows that the resulting noise still fits under the modulus that the flooding already forces.

LEMMA 5.2. *Let B_{bal} bound the noise of an accepted ballot and let a mix server pass Π_{Bnd} with parameter σ_{mix} . After n_M mixings the noise of every output ciphertext is bounded by $B_{\text{dec}} = B_{\text{bal}} + n_M \cdot 12\sigma_{\text{mix}}(1 + 2pkN)$, and the distributed decryption of [14] is correct and simulatable whenever $2^{\text{sec}} B_{\text{dec}} + B_{\text{dec}} < \lfloor q/2 \rfloor$.*

PROOF. Soundness of Π_{Bnd} yields for each i a vector $\bar{w}_i$ with $\|\bar{w}_i\|_\infty \leq 12\sigma_{\text{mix}}$ whose image under the mixing matrix is the published target, so the ciphertext added by the server is $c'_i = (A^T \bar{r}_i + p\bar{e}_{i,1}, t^T \bar{r}_i + p\bar{e}_{i,0})$ with all components bounded by $12\sigma_{\text{mix}}$. Lemma 5.1 with $f_0 = p\bar{e}_{i,0}$ bounds the noise of c'_i by $p\|\bar{e}_{i,0}\|_\infty + p \sum_{i \leq k} (\|e_i\|_2 \|\bar{r}_i\|_2 + \|s_i\|_2 \|\bar{e}_{1,i}\|_2)$, and replacing each ℓ_2 norm by $\sqrt{N}$ times the ℓ_∞ norm together with $\|e_i\|_\infty, \|s_i\|_\infty \leq 1$ gives $12\sigma_{\text{mix}}(1 + 2pkN)$. Noise is additive under ciphertext addition, so after n_M servers the total is the stated bound. The share of a trustee is $s_j^T u_i + pE_{i,j}$ with $\|E_{i,j}\|_\infty \leq 2^{\text{sec}} B_{\text{dec}}/(pt)$, so the combined value differs from the true noise by at most $2^{\text{sec}} B_{\text{dec}}$; below $\lfloor q/2 \rfloor$ no wraparound occurs and the reduction modulo p returns the plaintext, while the statistical distance between a real share and one sampled around the simulated value is at most $2^{-\text{sec}}$. $\square$

Because that modulus is the one the flooding already forces, dropping Π_{Small} costs nothing in the parameter set. The third lever reshapes the mixing commitment. A ciphertext has $k + 1$ components, so carrying them one by one needs $k + 1$ message slots and a width of at least $\eta + k + 3$. Oylama publishes the shuffled outputs first, derives h from SHAKE256 applied to the inputs and the outputs, and commits only to $\sum_{j \leq k} h^j c_{i,j}$, so the width falls to $\eta + 3$, which is 7 instead of 23 at $k = 16$. Since h is fixed after the outputs are, a server that mixes a component wrongly passes only if a nonzero polynomial of degree k vanishes at h in every residue slot, which happens with probability at most k/q_0 for q_0 the smallest prime factor of q .

The fourth lever batches the decryption phase. A trustee places the τ_0 flooding terms of a batch in the message slots of one commitment, derives short challenges $\chi_1, \dots, \chi_{\tau_0}$ from SHAKE256, and proves the single relation obtained from the χ_i weighted combination of the τ_0 relations $t_{i,j} = s_j^T u_i + pE_{i,j}$, so the data published per ciphertext and per trustee falls from $\eta + \ell + 1$ ring elements to two. The combined statement is again an instance of Π_{Lin} because the commitment is linearly homomorphic and the challenges are short, and a cheating trustee passes only if a random short combination of nonzero differences vanishes, with probability at most $|C|^{-1}$.

5.4 The Scheme

Algorithm 1 gives the four public procedures. Setup derives A from a SHAKE256 seed, runs the distributed key generation for (pk, s) with the replicated sharing of [22, 23], publishes one commitment per share, and generates the ML-DSA key of the registrar. Register(id) generates an ML-DSA key pair, publishes the verification key signed by the registrar and delivers the signing key over the hybrid channel. VerifyVote rebuilds the ciphertext from st_{pre} and checks that it is the last entry stored under id on the published board, so the check does not depend on which of two candidate votes the state holds. The recovery algorithm required by Definition 4.1 replaces each entry of BB that appears in BB_1 by the entry at the same position of BB_0 .

In Algorithm 1 the casting procedure takes the public data pd , the identity id with its key pair (upk, usk) and the option v ; it encodes

Algorithm 1 The four public procedures of Oylama

```

1: procedure VOTE( $pd, id, upk, usk, v$ )
2:    $\mu_v \leftarrow \text{Encode}(v); \quad (\mathbf{r}, \mathbf{e}_1, e_0) \leftarrow \{ -1, 0, 1 \}^{(2k+1)N}$ 
3:    $f_0 \leftarrow \mu_v + p e_0; \quad c \leftarrow (\mathbf{A}^\top \mathbf{r} + p \mathbf{e}_1, \mathbf{t}^\top \mathbf{r} + f_0)$ 
4:    $ctx \leftarrow \text{SHAKE256}(el, id, upk, c)$ 
5:    $\pi \leftarrow \Pi_{\text{Ball}}.\text{Prove}(c, (\mathbf{r}, \mathbf{e}_1, f_0), ctx)$ 
6:    $\sigma \leftarrow \text{ML-DSA}.\text{Sign}_{usk}(el, id, c, \pi)$ 
7:   return  $b = (c, \pi, \sigma)$ ,  $st_{\text{pre}} = (v, \mathbf{r}, \mathbf{e}_1, f_0)$ ,  $st_{\text{post}} = \text{SHAKE256}(b)$ 
8: end procedure
9: procedure VALID( $BB, id, b$ )
10:  if  $c \notin R_q^{k+1}$  or  $id \notin L$  then return  $\perp$ 
11:  if  $\text{ML-DSA}.\text{Vf}_{upk}(\cdot, \sigma) = \perp$  then return  $\perp$ 
12:  if  $\Pi_{\text{Ball}}.\text{Vf}(c, \pi, ctx) = \perp$  then return  $\perp$ 
13:  return  $\top$  if  $\text{SHAKE256}(b) \notin \{ \text{SHAKE256}(b') : b' \in BB \}$ 
14: end procedure

15: procedure TALLY( $BB, pd, sd$ )
16:   $BB^* \leftarrow$  last valid entry of each  $id$ , ordered by  $id$ 
17:   $C^{(0)} \leftarrow (c_i)_{i \leq \tau}$  from  $BB^*$ ,  $\tau = |BB^*|$ 
18:  for  $k = 1$  to  $n_M$  do
19:     $(C^{(k)}, \pi_k^{\text{mix}}) \leftarrow S_k.\Pi_{\text{Mix}}(C^{(k-1)})$ 
20:  end for
21:  for  $j \in$  quorum of size  $t$  do
22:     $ds_j \leftarrow T_j.\Pi_{\text{Dec}}(s_j, C^{(n_M)})$   $\triangleright$  shares,  $\Pi_{\text{Lin}}$  batched,  $\Pi_{\text{Bnd}}$ 
23:  end for
24:   $(\mu_i) \leftarrow \text{Comb}(C^{(n_M)}, \{ds_j\}); \quad \rho \leftarrow$  counts of  $\text{Decode}(\mu_i) \neq \perp$ 
25:  return  $\rho$ ,  $\Pi = (BB^*, \{\pi_k^{\text{mix}}\}, \{ds_j\})$ 
26: end procedure
27: procedure VERIFY( $pbb, \rho, \Pi$ )
28:  check  $BB^*$  against  $pbb$  and rerun VALID on each entry
29:  for  $k \leq n_M$ : check  $\Pi_{\text{Mix}}.\text{Vf}(C^{(k-1)}, C^{(k)}, \pi_k^{\text{mix}})$ 
30:  for  $j$ : check  $\Pi_{\text{Dec}}.\text{Vf}(C^{(n_M)}, ds_j)$ 
31:  return  $\top$  iff all checks pass and  $\rho$  matches Comb
32: end procedure

```

v , draws the ternary vectors, forms f_0 and the ciphertext c , binds the Fiat–Shamir context to the election identifier, the identity, the verification key and c so that a ballot cannot be moved to another voter, proves and signs, and returns the ballot with the private state and the digest that the voter looks for on the board. The validity procedure rejects a ciphertext outside R_q^{k+1} or an unregistered identity, verifies the signature and the proof, and accepts only a fresh digest, which blocks the replay of a whole ballot. The counting procedure keeps the last valid ballot of each identity, which is the revoting policy, passes the ciphertexts through the n_M shuffles, collects a share from each trustee of a quorum of size t , combines them, decodes while discarding plaintexts that fail the tag test, and outputs the counts with the transcript Π ; verification repeats every public step.

5.5 Parameters and Security

Table 2 lists the parameter set. The modulus is a product of three prime moduli congruent to 1 modulo $2N$, so every ring multiplication is a coefficientwise product in the transform domain, and its size follows from Lemma 5.2 and from the range of the Π_{Bnd} response on the flooding terms.

Symbol	Meaning	Value
N	degree of the base ring $R = \mathbb{Z}[X]/(X^N + 1)$	256
k	module rank, lattice dimension kN	16
q	ciphertext modulus, 3 NTT primes	$\approx 2^{94}$
p	plaintext modulus	2
χ	secret and noise distribution	ternary
κ	weight of a Fiat–Shamir challenge	23
sec	statistical parameter of the flooding	40
(η, ℓ, k_c)	commitment shape, mixing	(4, 1, 7)
(η, ℓ, k_c)	commitment shape, decryption key	(4, 16, 22)
τ_0	batch size of Π_{Bnd} and of Π_{Lin}	32
M	rejection sampling parameter	3
(t, n_T)	trustee threshold and count	(3, 4)
n_M	mix servers	2
ballot (c, π, σ)		71.8 KB
ML-KEM / ML-DSA instances		768 / 65

Table 2: Concrete Parameters of Oylama: the lattice, then the proof system, then the committee, then the measured ballot size and the standardized instances. The pair (kN, q) places the module problems above 128 bits according to the lattice estimator, the challenge weight κ makes the challenge space larger than 2^{131} , and sec controls the flooding of the decryption shares.

The two guarantees of Oylama are stated next. Theorem 5.3 addresses ballot privacy against a board chosen by the adversary and Theorem 5.4 addresses strong verifiability.

THEOREM 5.3. *Let SHAKE256 be modeled as a quantum random oracle. If $\text{MLWE}_{N,q,k,\chi}$ and $\text{MSIS}_{N,q,\eta,\beta}$ are hard, if ML-DSA is existentially unforgeable under chosen message attacks, if ML-KEM with AES-256-GCM is an indistinguishable hybrid encryption scheme under chosen ciphertext attacks, and if the parameters obey Lemma 5.2, then Oylama satisfies du-mb-BPRIV of Definition 4.1 with respect to the recovery algorithm of Section 5.4.*

THEOREM 5.4. *Let SHAKE256 be modeled as a quantum random oracle. If $\text{MSIS}_{N,q,\eta,\beta}$ is hard, if SHAKE256 is collision resistant against quantum adversaries, and if ML-DSA is existentially unforgeable under chosen message attacks, then Oylama satisfies strong verifiability of Definition 4.2.*

6 Computational Cost Analysis

We calculate the complexity of each procedure of each scheme of Section 2 from its own description, without using any figure published by its authors. For ring based schemes N is the degree of R , one ring multiplication costs $M = \Theta(N \log N)$ operations in $\mathbb{Z}_q$ through the number theoretic transform, one ring element occupies $|R_q| = N \lceil \log_2 q \rceil$ bits, k is the module rank and k_c the commitment width; for schemes over unstructured lattices v is the dimension and $m = \Theta(v \log q)$ the number of columns, so a matrix by vector product costs $\Theta(vm)$; for the multivariate scheme v is the number of variables and l the number of pixels of the cover image; $\lambda_S = \Theta(\lambda)$ is the number of Stern repetitions; and one rejection sampled response element occupies $N \log_2(6\sigma)$ bits.

Table 3 collects the result. Every scheme except Kyber-BC has a setup independent of n , because that scheme stores one key encapsulation key pair per voter. The three mix-net schemes pay $\Theta((n_M + t)n)$ ring operations and as many proof transcripts, while the homomorphic ones pay $\Theta(n)$ additions and one decryption at the price of a

ballot validity proof that grows with n_C . The four levers of Section 5.2 and Section 5.3 act on the mix-net family by the factors $2k/(k+1)$ on the ciphertext, $3k/(2k+1)$ on the ballot response, $(\eta+k+3)/(\eta+3)$ on the mixing commitment and its two Π_{Lin} openings, and τ_0 on the number of decryption proofs and commitments, so $2tnk_cN\log(6\sigma) + tn(\eta+\ell)|R_q|$ becomes $2t\lceil n/\tau_0 \rceil k_cN\log(6\sigma) + t\lceil n/\tau_0 \rceil (\eta+\tau_0)|R_q|$.

7 Implementations and Practical Comparison

All of our practical findings are available at the repository of the <https://github.com/ethanwaker9/work-36>. We conducted our experiments on an Apple MacBook Air (2022) with the M2 chip, A15 Bionic (64-bit ARM-based), an 8-core CPU featuring a base clock speed of 3.49 GHz, and 16GB of memory, Tahoe OS 26.2 build 25C56.

The Android application of Oylama is at https://github.com/ethanwaker9/work-36/blob/main/android_app/Oylama.apk and the web application is at https://github.com/ethanwaker9/work-36/tree/main/web_app. Figures 3 and 4 contain screenshots of our proof of concept implementations and we plan to deploy Oylama online soon.

Methodology. We implemented the cryptographic core of the thirteen schemes of Section 2 together with Oylama in one Python framework on top of numpy, with a single number theoretic transform layer, a single module commitment [68], a single verifiable shuffle [67] and a single verifiable distributed decryption [14]; the standardized primitives come from a compiled library, so ML-KEM, ML-DSA, SHAKE256 and AES-256-GCM run at reference speed. Each scheme keeps the structure described by its own authors, so one without an eligibility test or without a verifiable tally performs neither, and each is measured at parameters that place its own hard problem above 128 bits and that give every Fiat-Shamir proof a soundness error below 2^{-128} , which means one repetition over a ring or a module, seven with a twenty bit scalar challenge over an unstructured lattice, and 137 for the code based proof. Whenever a ballot is later decrypted by a threshold committee its modulus is the one that flooding forces, so the ballot column and the tally column of a scheme describe the same instance. Measurements use $n = 8$ ballots, two mix servers, four trustees of quorum three and two options, each time being the median over at least sixteen executions.

Table 4 presents the measured cost. Oylama is cheaper than Helios-PQ, NTRU-Mix and BPRIV-BC in all eight quantities and than Epoque in seven of them. Among the schemes that run a verifiable shuffle followed by a verifiable distributed decryption it spends 5.6 times less time per ballot on the tally and publishes a transcript 6.0 times smaller, which is the effect of the four levers of Section 5.3. Among the five schemes with a proved ballot privacy notion it has the smallest ballot and the fastest casting, by factors of 2.1 and 1.5 over the next of them. The entries below Oylama in the tally columns come from two groups. Kyber-BC, Hybrid-BC and Steg-MQPC publish no universal verification procedure, so their tally is a decryption loop; EVOLVE, EVOLVED and LWE-Voting avoid a decryption whose shares have to be flooded, which lets them work modulo 2^{31} or 2^{14} rather than 2^{94} , and they pay for it with the guarantees of the second column. No scheme whose privacy is proved with a simulator has a shorter tally.

Figure 3 makes the two families visible, since the schemes that omit the verifiable tally have a very short tally bar and a very small transcript bar, and among the mix-net schemes Oylama is the lowest

in every bar. Its two lower panels show that the tally and the verification follow the linear reference and that the transcript grows by a constant amount per ballot, so the cost of an election is predictable from a single measurement.

8 The Coercion Resistant Scheme $\widetilde{\text{Oylama}}$

8.1 Design

$\widetilde{\text{Oylama}}$ keeps the participants and the trust assumptions of Section 5.1 and adds an untappable channel from the registrar to the voter and an anonymous one from the voter to the board. A credential is a uniform $\text{cred} \in \{0,1\}^\lambda$, the roster holds one encryption of cred_i per voter, and FakeCred returns a fresh uniform string of the same length, so a coerced voter hands over an object the coercer cannot separate from a real credential. A ballot carries an encryption of the option and one of the credential with proofs of knowledge of both openings and is posted anonymously, so ballots with invalid credentials are accepted and removed later. This is the structure of [17] with the cleansing of [20], and the difficulty is to run that cleansing without a group of prime order.

Packed evaluation. Cleansing sorts the ballots and the roster by the pair formed by the credential and the board position, marks an entry valid when its successor carries the same credential and comes from the roster, and replaces the option of an invalid entry by an encryption of an invalid option. In [20] each binary gate of that computation is a conditional gate costing one threshold decryption and a proof. $\widetilde{\text{Oylama}}$ keeps the circuit and evaluates it on packed plaintexts: with a prime $p' \equiv 1 \pmod{2N}$ the ring $R_{p'}$ splits into N slots and one ciphertext carries N independent bits. Addition and multiplication by a public plaintext are local. Multiplication of two encrypted bit vectors uses a masked opening, where the trustees publish encryptions of uniform masks α_j , open $x + \sum_j \alpha_j$ through the verifiable distributed decryption and return $\tilde{x} \cdot \llbracket y \rrbracket - \sum_j \alpha_j \cdot \llbracket y \rrbracket$; each product multiplies by a public or locally known plaintext and is proved with one instance of Π_{Lin} because it is linear in the committed mask. Data movement uses the automorphism $X \mapsto X^{3^t}$, which permutes the slots cyclically: the image of a ciphertext decrypts under the image of the key, which each trustee obtains from its own share, and the conversion back to the base key happens inside the next masked opening at the price of one proof that two ciphertexts under two public keys carry the same plaintext. Since a masked opening treats all N slots at once, one distributed decryption serves N parallel gates, which is the source of the gain over [20].

Cost. Sorting $M = n_B + n_V$ entries with keys of $L = \lambda + m$ bits by Batchier's network takes $\frac{1}{2} \log M (\log M + 1)$ stages, each needing $3 + \log B$ packed multiplications and $5 + 2 \log B$ packed rotations for B bit blocks, and the validity step adds $2 \log B + 2$ multiplications; with $\lceil MB/(N/2) \rceil$ ciphertexts in flight the cleansing costs $O(\lceil MBN^{-1} \rceil \log^2 M \log B)$ distributed decryptions instead of the $O(M \log^2 M (L + \lambda))$ conditional gates of [20].

8.2 Security and Performance

Theorem 8.1 addresses coercion resistance and Theorem 8.2 strong verifiability; both proofs are in the appendix of the full version. Definition 4.3 implies ballot privacy and receipt freeness, because an

Scheme	Setup (time / space)	Vote (time / space)	Valid (time)	Tally (time / space)	Verify (time)
Epoque [8]	$\Theta(vm) / \Theta(v \log q)$	$\Theta(vm) / \Theta(m \log q)$	$\Theta(vm)$	$\Theta(nvm + n_C n_T \log n) / \Theta(n \log q)$	$\Theta(nvm)$
NTRU-Mix [9]	$\Theta(M) / \Theta(R_q)$	$\Theta(M) / \Theta(R_q)$	$\Theta(M)$	$\Theta((n_M + t)nkM) / \Theta((n_M + t)n(R_q + kN \log 6\sigma))$	$\Theta((n_M + t)nkM)$
FOO-based [36]	$\Theta(vm) / \Theta(v \log q)$	$\Theta(M_{rej}M + vm) / \Theta(N \log 6\sigma + m \log q)$	$\Theta(M)$	$\Theta(tnmv) / \Theta(tnv \log q)$	$\Theta(nM)$
Code-based [43]	$\Theta(vm) / \Theta(vm)$	$\Theta(\lambda_S vm) / \Theta(\lambda_S(v + m))$	$\Theta(\lambda_S vm)$	$\Theta(nvm) / \Theta(nv)$	$\Theta(n\lambda_S vm)$
EVOLVE [7]	$\Theta(kM) / \Theta(k R_q)$	$\Theta(n_C kM) / \Theta(n_T R_q + n_C kN \log 6\sigma)$	$\Theta(n_C kM)$	$\Theta(nM + n_T kM) / \Theta(n_T kN \log 6\sigma)$	$\Theta(nn_C kM)$
EVOLVED [45]	$\Theta(kM) / \Theta(k R_q)$	$\Theta((n_C + \log n_C)kM) / \Theta(n_T R_q + n_C kN \log 6\sigma)$	$\Theta(n_C kM)$	$\Theta(nM + n_T kM) / \Theta(n_T kN \log 6\sigma)$	$\Theta(nn_C kM)$
Homo-LWE [6]	$\Theta(M) / \Theta(R_q)$	$\Theta(M) / \Theta(R_q)$	$\Theta(M)$	$\Theta(nM + LM) / \Theta(L R_q)$	$\Theta(nM)$
BPRIV-BC [49]	$\Theta(kM) / \Theta(k R_q)$	$\Theta(kM) / \Theta(R_q + kN \log 6\sigma)$	$\Theta(kM)$	$\Theta(nM) / \Theta(n R_q)$	$\Theta(nkM)$
Kyber-BC [54]	$\Theta(n) / \Theta(n\lambda)$	$\Theta(1) / \Theta(\lambda)$	$\Theta(1)$	$\Theta(n) / \Theta(n\lambda)$	–
Hybrid-BC [59]	$\Theta(1) / \Theta(\lambda)$	$\Theta(1) / \Theta(\lambda)$	$\Theta(1)$	$\Theta(n) / \Theta(n\lambda)$	–
Steg-MQPC [63]	$\Theta(v^3) / \Theta(v^3)$	$\Theta(v^3 I) / \Theta(I)$	$\Theta(1)$	$\Theta(n(v^3 I)) / \Theta(nv)$	–
LWE-Voting [46]	$\Theta(vm) / \Theta(m \log q)$	$\Theta(vm) / \Theta(v \log q)$	$\Theta(1)$	$\Theta(nv) / \Theta(v \log q)$	$\Theta(n)$
Helios-PQ [66]	$\Theta(kM) / \Theta(k R_q)$	$\Theta(kM) / \Theta(R_q + kN \log 6\sigma)$	$\Theta(kM)$	$\Theta((n_M + t)nkM) / \Theta((n_M + t)n(R_q + kN \log 6\sigma))$	$\Theta((n_M + t)nkM)$
Oylama	$\Theta(k^2 N) / \Theta(k R_q)$	$\Theta(k^2 N + kM) / ((k+1) R_q + (2k+1)N \log 6\sigma)$	$\Theta(k^2 N + kM)$	$\Theta((n_M n + t \lceil n / \tau_0 \rceil)kM) / \Theta(n_M n(k + \eta + 2) R_q + t n R_q + (n_M n + t \lceil n / \tau_0 \rceil)k_C N \log 6\sigma)$	$\Theta((n_M n + t \lceil n / \tau_0 \rceil)kM)$

Table 3: Time and space complexity of the five procedures, derived from the description of each scheme, with n ballots, n_C options, n_T trustees of quorum t , n_M mix servers, batch size τ_0 , ring degree N , ring multiplication cost $M = \Theta(N \log N)$, ring element size $|R_q| = N \lceil \log_2 q \rceil$, commitment width k , unstructured dimensions v and $m = \Theta(v \log q)$, homomorphic depth L , expected rejection repetitions M_{rej} , Stern repetitions λ_S , image size I and response deviation σ . A dash marks a scheme with no universal verification procedure. The last row shows where Oylama differs from the other mix-net schemes, since it uses a module of rank k over a base ring of degree N rather than a ring of degree kN , a three element ballot opening rather than four, a mixing commitment with one message slot rather than $k+1$, and $t \lceil n / \tau_0 \rceil$ linear proofs in the decryption phase rather than tn .

Scheme	Privacy / Verifiability	Setup		Vote		Valid	Tally		Verify
		ms	KB	ms	KB	ms	ms	KB	ms
Epoque [8]	wBPRIV / i,u	14.8	12802	66.7	350	43.2	59.2	306	116
NTRU-Mix [9]	– / u	156	329	11.6	121	4.49	166	1397	15.4
FOO-based [36]	– / pu	14.3	12806	22.1	30.0	0.74	45.6	306	59.1
Code-based [43]	– / i,u	3.49	255	30.7	763	20.1	25.0	763	45.9
EVOLVE [7]	BPRIV / pu	4.70	64.9	15.5	191	3.38	0.01	0.78	3.42
EVOLVED [45]	– / pu	4.51	64.9	20.0	243	4.40	0.01	0.78	4.52
Homo-LWE [6]	BPRIV* / i,u	192	183	53.4	452	17.6	4.68	52.0	18.3
BPRIV-BC [49]	BPRIV / i,u	113	315	17.0	148	6.41	101	513	16.1
Kyber-BC [54]	– / pi	0.28	29.9	0.13	4.38	0.05	0.06	0.03	–
Hybrid-BC [59]	– / pi	30.3	28801	18.3	114	0.00	0.06	0.03	–
Steg-MQPC [63]	– / –	1.60	436	0.25	256	0.00	0.00	0.09	–
LWE-Voting [46]	– / u	2.35	3.25	10.2	49.2	2.66	13.0	43.4	19.0
Helios-PQ [66]	– / –	108	331	17.2	131	5.27	169	1398	16.2
Oylama	du-mb / str	15.8	225	10.0	71.8	3.90	29.6	234	14.8

Table 4: Measured cost of the five procedures for eight ballots, two mix servers, four trustees of quorum three and two options, on one core; times are milliseconds per ballot and sizes kilobytes, with the setup column giving the public election data, the vote column one ballot and the tally column the transcript per ballot. The second column states the proven notions, where wBPRIV is the weak notion of [30], i, u, pi and pu are individual, universal, partial individual and partial universal verifiability, du-mb is Definition 4.1, str is Definition 4.2, a dash is the absence of a stated guarantee and an asterisk a proof sketch. A dash in the verify column marks a scheme with no universal verification procedure, and a tally entry of a scheme whose verifiability column carries a dash measures a plain decryption loop.

adversary against either notion is a coercer that never interrogates the voter or accepts the material without instructing a vote, and the ideal experiment reveals nothing beyond the result.

THEOREM 8.1. *Let SHAKE256 be modeled as a quantum random oracle. If $\text{MLWE}_{N,q,1,\chi}$ and $\text{MSIS}_{N,q,\eta,\beta}$ are hard, if at most $t-1$ trustees are corrupted, if the registration channel is untappable and the submission channel is anonymous, and if the parameters satisfy the noise condition of Lemma 5.2 for the packed plaintext modulus p' , then Oylama satisfies coercion resistance in the sense of Definition 4.3.*

THEOREM 8.2. *Under the assumptions of Theorem 8.1 and the collision resistance of SHAKE256 against quantum adversaries, Oylama satisfies strong verifiability of Definition 4.2.*

We implemented Oylama with $N = 8192$, five prime moduli of 24 bits, packed plaintext modulus $p' = 65537$, credentials of $\lambda = 128$ bits, three trustees of quorum two and two options, and checked the cleansing against a reference computation in the clear on every instance.

The three measured points give a cleansing of 129.9, 213.8 and 322.4 seconds for $M = 4, 8, 16$, with 52, 85 and 129 packed gate batches, a ballot of 685 KB and a casting time of about 0.1 second per voter, and the cleansing recovers the cast options on every instance. The growth between consecutive points, 1.65 and 1.51, matches the ratio 1.63 and 1.52 predicted by the gate count, so the cost model describes the implementation well.

A Appendix

This appendix contains the material that the nine page version omits, which is the two experiments of the coercion resistance definition, a diagram of the placement of the primitives, the derivation of the parameter set, and the complete proofs of Theorems 5.3, 5.4, 8.1 and 8.2.

A.1 The Coercion Resistance Experiments

Figure 5 states the two experiments referred to by Definition 4.3.

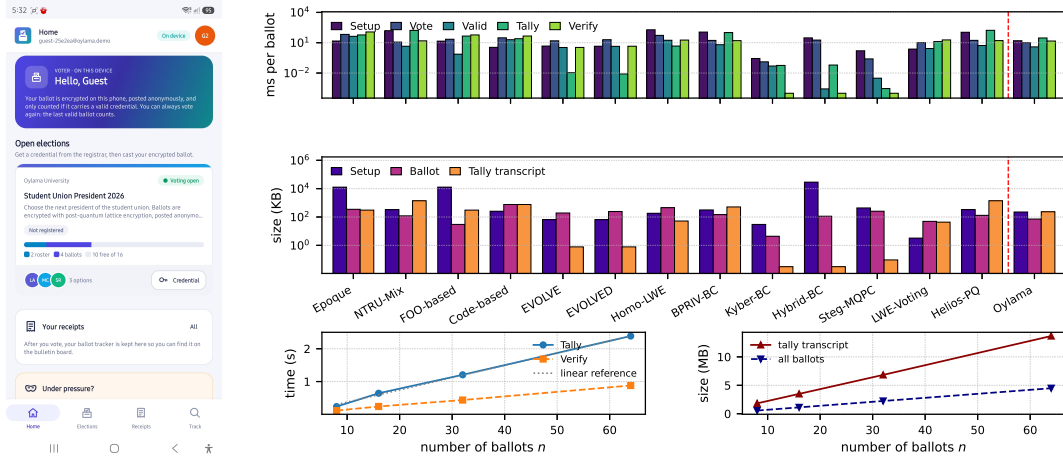


Figure 3: Cost of the fourteen schemes on logarithmic scales, for eight ballots, two mix servers and four trustees of quorum three. The first panel gives the time of the five procedures in milliseconds per ballot, with the bars of a group ordered setup, vote, valid, tally, verify; the second gives the size of the public setup data, of one ballot and of the tally transcript per ballot in kilobytes; the dashed line separates Oylama. The last two panels give the scaling of Oylama with the number of ballots n , in wall-clock seconds against a linear reference through the last point and in megabytes. A short tally or verify bar means the scheme performs no verifiable counting, and among the mix-net schemes the tally bar of Oylama is the shortest.

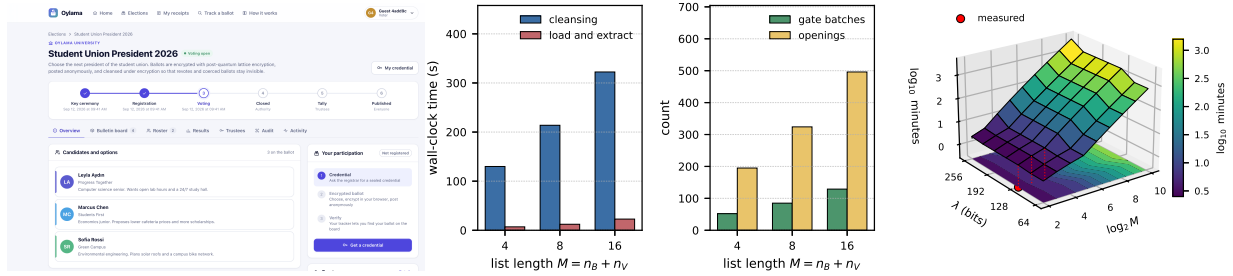


Figure 4: Cost of Oylama for lists of $M = 4, 8, 16$ entries, an entry being a ballot or a roster record. Left: wall-clock time of the cleansing and of loading and extraction. Middle: number of packed gate batches and of verifiable openings. Right: cleansing time against $\log_2 M$ and the credential length λ , with height and color the base ten logarithm of the time in minutes, from the model of Section 8.1 whose two constants are fitted to the three measured points in red. The gate count grows like $\log^2 M$ because the network has $\frac{1}{2} \log M (\log M + 1)$ stages, and the surface steps upward when λ crosses a power of two.

A.2 Placement of the Primitives

Figure ?? shows where each standardized primitive sits among the participants of Section 5.1. The registrar reaches the voter through a key encapsulation whose shared secret keys an authenticated cipher, the voter reaches the board with a ciphertext, a proof and a signature, the board is under the control of the adversary and only forwards its public view, the mix servers act in sequence on the ciphertexts, the trustees hold the shares of the decryption key, and the auditors recompute every public check.

A.3 Derivation of the Parameter Set

We derive the values of Table 2. Write $B_\chi = 1$ for the infinity norm of a ternary sample, $\kappa = 23$ for the challenge weight and $\text{sec} = 40$ for the statistical parameter. A ballot proof uses the witness (r, e_1, f_0) of dimension $(2k+1)N = 8448$ with $\|(r, e_1, f_0)\|_\infty \leq p+1 = 3$, so rejection sampling with $M = 3$ asks for $\sigma_{\text{bal}} = 11 \cdot 3 \cdot \sqrt{\kappa(2k+1)N} \approx 2^{13.8}$ and the extractor of Section A.4 returns a witness with infinity norm at most

$12\sigma_{\text{bal}} \approx 2^{17.4}$. By Lemma 5.1 the noise of an accepted ballot obeys

$$B_{\text{bal}} \leq 12\sigma_{\text{bal}}(1+2pkN) \approx 2^{31.4}.$$

A mixing batch of $\tau_0 = 32$ ciphertexts uses a witness of dimension $\text{cols} \cdot N$ with $\text{cols} = k_c + 2k + 1 = 40$, so $\sigma_{\text{mix}} = 11\sqrt{\kappa \text{cols} N \tau_0} \approx 2^{14.9}$ and, by Lemma 5.2, the noise after n_M servers is at most $B_{\text{dec}} = B_{\text{bal}} + n_M \cdot 12\sigma_{\text{mix}}(1+2pkN) \approx 2^{33.8}$ for $n_M = 2$. Each of the t trustees draws a flooding term bounded by $2^{\text{sec} B_{\text{dec}}/(pt)} \approx 2^{71.2}$, and the batched commitment of Section 5.3 places τ_0 of them in one statement of dimension $(\eta + 2\tau_0 + 2)N = 17920$, so the response of Π_{Bnd} on those terms has deviation $\sigma_E \approx 2^{83.9}$ and infinity norm at most $12\sigma_E \approx 2^{87.5}$. Correctness of the combination asks for $pt \cdot 12\sigma_E < q$, which is the binding constraint here and gives $\log_2 q \geq 92$. Three prime moduli of 31 bits congruent to 1 modulo $2N$ give $\log_2 q = 94$, which leaves two bits of margin, and the pair $(kN, q) = (4096, 2^{94})$ is above 128 bits for MLWE and MSIS with a ternary distribution according to the lattice estimator [48] and to the parameter tables of [86]. The challenge set $C = \{c \in R : \|c\|_\infty \leq 1, \|c\|_1 = \kappa\}$ has $\binom{N}{\kappa} 2^\kappa > 2^{131}$ elements and its differences are invertible in R_p for the split of $X^N + 1$ that p induces, which is

$\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Real}}(\lambda, n_T, t, n_V, n_A, n_C, \mathcal{B})$ $\begin{aligned} & BB \leftarrow \emptyset \\ & (pk, \{(s_i, h_i)\}_{i=1}^{n_T}) \leftarrow \mathcal{P}_{\text{Setup}}^{\mathcal{A}}(\lambda, n_T, t) \\ & \{\text{cred}_i\}_{i=1}^{n_V}, R \leftarrow \text{Register}(\lambda, pk, n_V) \\ & \mathcal{D} \leftarrow \mathcal{A}(R) \\ & \text{if } \mathcal{D} \neq n_A \text{ then return } 0 \\ & (j, \alpha) \leftarrow \mathcal{A}(\{\text{cred}_i\}_{i \in \mathcal{D}}) \\ & \text{if } j \in \mathcal{D} \vee \alpha \notin [1, n_C] \cup \{\phi\} \text{ then return } 0 \\ & B \leftarrow \mathcal{B}([1, n_V] \setminus \mathcal{D}, n_C) \\ & \text{for } (i, v) \in B \text{ with } i \notin [1, n_V] \text{ do} \\ & \quad \text{cred}_i \leftarrow \text{FakeCred}(pk) \\ & \beta \leftarrow \mathcal{S}\{0, 1\} \\ & \widetilde{\text{cred}} \leftarrow \text{cred}_j \\ & \text{if } \beta = 0 \text{ then} \\ & \quad \widetilde{\text{cred}} \leftarrow \text{FakeCred}(\text{cred}_j) \\ & \quad \text{remove all } (j, \star) \text{ from } B \text{ but the last,} \\ & \quad \text{which is replaced by } (j, \alpha) \text{ if } \alpha \neq \phi, \\ & \quad \text{else removed} \\ & \text{else} \\ & \quad \text{remove all } (j, \star) \text{ from } B \\ & \text{for } (i, v) \in B \text{ in order do} \\ & \quad M \leftarrow \mathcal{A}^{\mathcal{O}_{\text{board}}}(BB) \\ & \quad BB \leftarrow BB \parallel \{m \in M \mid \text{Valid}(pk, m, BB) = \top\} \\ & \quad b \leftarrow \text{Vote}(pk, \text{cred}_i, v) \\ & \quad BB \leftarrow BB \parallel b \\ & M \leftarrow \mathcal{A}^{\mathcal{O}_{\text{board}}}(BB, \widetilde{\text{cred}}, \text{"casting closed"}) \\ & BB \leftarrow BB \parallel \{m \in M \mid \text{Valid}(pk, m, BB) = \top\} \\ & (r^*, \Pi^*) \leftarrow \mathcal{P}_{\text{Tally}}^{\mathcal{A}}(BB, R, pk, \{(s_i, h_i)\}, t) \\ & d \leftarrow \mathcal{A}(r^*, \Pi^*) \\ & \text{return } d \end{aligned}$	$\mathcal{O}_{\text{board}}()$ $1: \text{return Publish}(BB)$ $\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Ideal}}(\lambda, n_V, n_A, n_C, \mathcal{B})$ $\begin{aligned} & \mathcal{D} \leftarrow \mathcal{A}(\lambda) \\ & \text{if } \mathcal{D} \neq n_A \text{ then return } 0 \\ & (j, \alpha) \leftarrow \mathcal{A}() \\ & \text{if } j \in \mathcal{D} \vee \alpha \notin [1, n_C] \cup \{\phi\} \text{ then return } 0 \\ & B \leftarrow \mathcal{B}([1, n_V] \setminus \mathcal{D}, n_C) \\ & \beta \leftarrow \mathcal{S}\{0, 1\} \\ & \text{if } \beta = 0 \text{ then} \\ & \quad \text{remove all } (j, \star) \text{ from } B \text{ but the last,} \\ & \quad \text{which is replaced by } (j, \alpha) \text{ if } \alpha \neq \phi, \\ & \quad \text{else removed} \\ & \text{else} \\ & \quad \text{remove all } (j, \star) \text{ from } B \\ & \text{for } (i, v) \in B \text{ with } i \in [1, n_V] \cup \{\text{neg}\} \text{ do} \\ & \quad D \leftarrow D \parallel (i, v) \\ & (\{v_i\}_{i \in \mathcal{D}}, \gamma) \leftarrow \mathcal{A}(D) \\ & \text{for } i \in \mathcal{D} \text{ do} \\ & \quad \text{if } v_i \in [1, n_C] \text{ then } D \leftarrow D \parallel (i, v_i) \\ & \text{if } \beta = 1 \wedge \gamma \in [1, n_C] \text{ then} \\ & \quad D \leftarrow D \parallel (j, \gamma) \\ & r^* \leftarrow \text{Result}(\text{Cleanse}(D)) \\ & d \leftarrow \mathcal{A}(r^*) \\ & \text{return } d \end{aligned}$
--	---

Figure 5: In the Real experiment the adversary is given the disclosed credential $\widetilde{\text{cred}}$ during the casting phase, may keep posting ballots on its behalf until casting closes, and finally observes the published tally and proof. In the Ideal experiment, only the board length $|D|$, the corrupted-voter assignments, and the tally are revealed; the disclosed credential is not modeled because no post-tally interrogation is permitted.

what the extraction of Section A.4 needs. A ballot then occupies $(k + 1)N \lceil \log_2 q \rceil$ bits for the ciphertext, $(2k + 1)N \log_2(6\sigma_{\text{bal}})$ bits for the response and 3309 bytes for the signature, which is 71.8 KB in total.

A.4 Proof of Theorem 5.3

Throughout, q_H bounds the number of quantum queries that $\mathcal{A}$ makes to SHAKE256, q_V the number of queries to $\mathcal{O}_{\text{voteLR}}$ and τ the number of ballots that survive the weeding step. We first describe the simulator and then bound the advantage by a sequence of games.

The simulator. On input (pd, PBB, ρ) the algorithm Sim proceeds as follows. It parses PBB into entries $(id, upk, c, \pi, \sigma)$, applies the weeding rule of Algorithm 1 to obtain the list BB^* and the ciphertext vector $C^{(0)}$. For $k = 1, \dots, n_M$ it runs the honest mixing algorithm Π_{Mix} on $C^{(k-1)}$ with a permutation and re-randomization randomness of its own choice, which requires no secret material, and obtains $C^{(k)}$ together with π_k^{mix} . It then fixes a list of plaintexts $(\mu_i)_{i \leq \tau}$ whose decoded values form the multiset ρ , padding with plaintexts that fail the tag test of Section 5.2 when τ exceeds the number of counted votes. For each trustee j of the quorum it runs the simulator $S_{\Pi_{\text{Dec}}}$ of the verifiable distributed decryption on input $(C^{(n_M)}, (\mu_i))$ and the

public commitment to the share of j , obtaining a transcript ds_j . It outputs $\Pi = (BB^*, \{\pi_k^{\text{mix}}\}, \{ds_j\})$.

The algorithm $S_{\Pi_{\text{Dec}}}$ exists and its output is close to a real transcript. Indeed a real transcript consists of the values $t_{i,j} = s_j^\top \mathbf{u}_i + pE_{i,j}$, of one commitment per batch that carries the τ_0 terms $E_{i,j}$ of that batch, of one instance of Π_{Lin} per batch and of one instance of Π_{Bnd} per batch. Given the target plaintexts, $S_{\Pi_{\text{Dec}}}$ samples $t_{i,j}$ for the $t-1$ possibly corrupted trustees exactly as the protocol does and fixes the remaining share so that the combination yields μ_i ; it commits to arbitrary values in place of $E_{i,j}$ and simulates the two proofs by programming the quantum random oracle at the points determined by the first messages. By Lemma 5.2 the real $t_{i,j}$ of an honest trustee is a uniform element of a range of size $2^{\text{sec}+1} B_{\text{dec}} / (pt)$ shifted by a value of magnitude at most B_{dec} , so the statistical distance between the real and the simulated value is at most $2^{-\text{sec}}$ per ciphertext and per trustee. The commitments are indistinguishable by the hiding property, which reduces to MLWE, and the two proofs are indistinguishable by the adaptive reprogramming bound of [89], which costs $O(q_H \sqrt{q_H} / \sqrt{|\mathcal{R}|})$ where $\mathcal{R}$ is the range of the first message.

Game G_0 . This is $\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, \text{Recover}}^{\text{du-mb-BPRIV}, 0}(\lambda)$.

Game G_1 . As G_0 , except that O_{tally} returns $(\rho, \text{Sim}(pd, \text{Publish}(BB), \rho))$ instead of the transcript produced by the honest trustees. The mixing part of the transcript is produced by the same randomized algorithm in both games and is therefore identically distributed. The decryption part differs, and the discussion above bounds the difference, so

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \tau t 2^{-\text{sec}} + t \varepsilon_{\text{hide}} + t \varepsilon_{\text{zk}},$$

where $\varepsilon_{\text{hide}}$ is the advantage against the hiding property of the commitment and ε_{zk} the reprogramming term.

Games $H_0, \dots, H_{q_V}$. For $\ell \in \{0, \dots, q_V\}$ let H_ℓ be the game in which O_{board} answers with the board whose first ℓ honest entries are the ballots computed for v_1 and whose remaining honest entries are the ballots computed for v_0 , while O_{voteLR} stores st_{pre} of the entry that the board shows, and O_{tally} computes the result on the board obtained by replacing each of the first ℓ honest entries by its v_0 counterpart and returns Sim applied to the published board and to that result. Then $H_0 = G_1$ and $H_{q_V} = \text{Exp}^{\text{du-mb-bpriv}}_{\mathcal{A}, \mathcal{V}, \text{Sim}}(\lambda)$, because the recovery algorithm of Section 5.4 performs exactly that replacement and because VerifyVote rebuilds the ciphertext from st_{pre} , so the set *Happy* is the same in the two extreme games.

Bounding one step. Fix ℓ and let $\mathcal{D}$ be the following distinguisher for the indistinguishability of BGV ciphertexts. On input a public key pk and a challenge ciphertext c^* that encrypts either μ_{v_0} or μ_{v_1} , the algorithm $\mathcal{D}$ runs Setup on pk while generating the credentials, the registrar key and the board key itself, and simulates every oracle as in $H_{\ell-1}$, except that on the ℓ -th call to O_{voteLR} it embeds c^* in the ballot and produces the proof π with the zero-knowledge simulator, again by programming the oracle. When $\mathcal{A}$ outputs a board BB , the algorithm $\mathcal{D}$ has to compute the result without the decryption key. It does so as follows. Every entry of BB that passes Valid carries a signature that verifies under a registered verification key, so by the existential unforgeability of ML-DSA an entry stored under an honest identity either equals a ballot returned by O_{voteLR} , whose vote $\mathcal{D}$ knows, or breaks the signature scheme. For an entry stored under a corrupted identity, $\mathcal{D}$ runs the online extractor of the measure-and-reprogram technique [91] on the proof π of that entry and obtains, with probability $1 - O(q_H^2) \varepsilon_{\text{ss}}$, a triple $(\bar{r}, \bar{e}_1, \bar{f}_0)$ and a difference of challenges $\bar{d}$ such that $\bar{d}u = \bar{a}\bar{r} + p\bar{e}_1$ and $\bar{d}w = \bar{b}\bar{r} + \bar{f}_0$ in R_q . By Lemma 5.1 applied to $\bar{d} \cdot c$, the plaintext of $\bar{d} \cdot c$ is $\bar{f}_0 \bmod p$, and since $\bar{d}$ is a difference of two distinct challenges it is invertible in R_p by the choice of C made in Section A.3, so the plaintext of c is $\bar{d}^{-1} \bar{f}_0 \bmod p$. Decoding that plaintext gives the vote of the entry or the symbol $\perp$, and $\mathcal{D}$ discards the latter, which is what the honest tally does. Adding the votes of the honest entries that appear on BB , taken from the left board as the recovery prescribes, $\mathcal{D}$ obtains the same result ρ as the experiment. It then calls Sim , answers O_{verify} by comparing the board entries to the ciphertexts it holds, and outputs whatever the experiment outputs.

When c^* encrypts μ_{v_1} the view of $\mathcal{A}$ is that of H_ℓ and when it encrypts μ_{v_0} it is that of $H_{\ell-1}$, up to the replacement of the real proof π of the ℓ -th ballot by a simulated one, which costs one more reprogramming term. Since the result ρ is unchanged by construction, the simulated transcript is unchanged as well. The indistinguishability of two BGV ciphertexts under the same key follows from $\text{MLWE}_{N, q, 1, \chi}$ by the standard argument that replaces (A, t) by a uniform pair and then (u, w) by a uniform pair. Hence

$$|\Pr[H_{\ell-1} = 1] - \Pr[H_\ell = 1]| \leq \varepsilon_{\text{MLWE}} + \varepsilon_{\text{euf}} + O(q_H^2) \varepsilon_{\text{ss}} + \varepsilon_{\text{zk}}.$$

Conclusion. Summing the q_V steps and the first hop,

$$\text{Adv}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{du-mb-bpriv}}(\lambda) \leq \tau t 2^{-\text{sec}} + t \varepsilon_{\text{hide}} + (t + q_V) \varepsilon_{\text{zk}} + q_V (\varepsilon_{\text{MLWE}} + \varepsilon_{\text{euf}} + O(q_H^2) \varepsilon_{\text{ss}}),$$

which is negligible under the stated assumptions. $\square$

A.5 Proof of Theorem 5.4

We treat the two experiments of Definition 4.2 in turn. In both we use the following three claims, which hold whenever $\text{Verify}(BB, \rho, \Pi) = 1$.

Claim 1. With probability at least $1 - n_M \varepsilon_{\text{MSIS}} - n_M \tau / |C| - n_M k \tau / q_0$ the multiset of plaintexts of $C^{(n_M)}$ equals the multiset of plaintexts of $C^{(0)}$. Indeed the verification of Π_{Mix} derives h from SHAKE256 applied to the inputs and to the published outputs, folds each ciphertext into $\sum_{j \leq k} h^j c_{i,j}$, and runs Π_{Shuf} on the commitments to the folded inputs and on the folded outputs; Π_{Shuf} is a proof of knowledge for the relation that the committed values are a permutation of the opened ones, whose soundness follows from the binding property of the commitment, which reduces to MSIS, and from the product argument of Aranha et al. [67]. Two ciphertext vectors with the same fold differ on some component only if a nonzero polynomial of degree k vanishes at h , which happens with probability at most k/q_0 at each of the τ positions because h is drawn after the outputs are fixed, where q_0 is the smallest prime factor of q . The verification also runs Π_{Bnd} on the committed re-randomization randomness, so each opened value is $c_i + \text{Enc}_{pk}(0; r'_i)$ for a vector of infinity norm at most $12\sigma_{\text{mix}}$, which does not change the plaintext.

Claim 2. With probability at least $1 - t(\varepsilon_{\text{MSIS}} + 1/|C|)$ the value returned by Comb on the accepted shares equals the plaintext vector of $C^{(n_M)}$. The verification checks, for each trustee and each batch, one instance of Π_{Lin} for the relation $t_{i,j} = s_j^\top u_i + pE_{i,j}$ aggregated with short challenges χ_i and one instance of Π_{Bnd} for the bound on the whole batch of terms $E_{i,j}$. If some individual relation fails while the aggregated one holds, then a nonzero short combination of the differences vanishes, which happens with probability at most $1/|C|$ over the choice of the χ_i ; and the committed share s_j is the one fixed at key generation because the commitment is binding. Given the bound of Lemma 5.2 the combination does not wrap around modulo q , so the reduction modulo p returns the plaintext.

Claim 3. Every ciphertext of $C^{(0)}$ comes from an entry of BB that passes Valid , and Valid accepts only entries whose signature verifies under a verification key that appears in the public credential list L and whose digest is new. Consequently the map from entries of $C^{(0)}$ to identities is well defined and injective after the weeding step, which keeps one entry per identity.

Experiment $\text{Exp}^{\text{ver-b}}$. Here the registrar is honest, so every key in L was produced by O_{reg} and the signing key of an identity outside CU is unknown to $\mathcal{A}$. Suppose $\mathcal{A}$ wins. By Claims 1 to 3 the announced result is $\rho = \rho(\{\text{Decode}(\mu_i)\})$ where the μ_i are the plaintexts of the ciphertexts of the entries kept by the weeding step, one per identity. Partition those identities into the checked honest ones, the remaining honest ones and the corrupted ones. For id in *Checked* the predicate CheckedVoter holds, so by the definition of VerifyVote the ciphertext rebuilt from st_{pre} is the last entry under id , hence the kept entry, hence its vote v_{id}^E is counted. For id honest and not checked the kept entry is either a ballot returned by O_{vote} , in which case its

vote belongs to $HVote \setminus Checked$, or a forgery under an honest key, which contradicts the existential unforgeability of ML-DSA. For id corrupted at most one entry is kept, so at most $|CU|$ further votes enter the multiset. Since ρ admits partial tallying, the announced result therefore has the form $\rho(\{v_l^E\}) \oplus_R \rho(\{v_l^A\}) \oplus_R \rho(\{v_l^B\})$ with $|\{v_l^B\}| \leq |CU|$, so the experiment returns 0, which contradicts the assumption. Collecting the failure probabilities gives

$$\Pr[\text{Exp}^{\text{ver-b}} = 1] \leq (n_M + t)\varepsilon_{\text{MSIS}} + \frac{n_M \tau + t}{|C|} + \frac{n_M k \tau}{q_0} + q_V \varepsilon_{\text{euf}} + \varepsilon_{\text{cr}},$$

where ε_{cr} bounds the probability of a collision of SHAKE256 against a quantum adversary, which is needed because the weeding step compares digests.

Experiment $\text{Exp}^{\text{ver-r}}$. Here the bulletin board is honest, so BB contains exactly the entries submitted through O_{cast} , in the order in which they were submitted, and the weeding step is performed by the honest board. The adversary controls the registrar, so it may place arbitrary keys in L ; however the ballots of honest voters are produced by O_{vote} with keys that the adversary does not choose for those voters, and a checked voter has confirmed that its own ballot is the last entry under its identity. Claims 1 to 3 apply unchanged because they use only the soundness of the proofs and the acceptance rule of Valid. The same partition of the kept entries gives the same decomposition of ρ , so the experiment returns 0 except with the probability displayed above. $\square$

A.6 Proof of Theorem 8.1

Let $\mathcal{A}$ be a coercer in the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Real}}$ of Definition 4.3 for parameters n_T, t, n_V, n_A, n_C and a behavioral distribution $\mathcal{B}$. We build $\mathcal{B}^*$ through a sequence of games whose first is the real experiment and whose last is a game that $\mathcal{B}^*$ can run inside the ideal experiment. Below, Θ denotes the total number of packed openings that the cleansing performs, which is the quantity plotted in the middle panel of Figure 4.

Game C_0 . The real experiment.

Game C_1 . Every non-interactive proof produced by an honest party is replaced by the output of its simulator, which programs the quantum random oracle at the point determined by the first message. The proofs concerned are the two ballot proofs of each voter, the proof of correct plaintext multiplication of each trustee in every masked opening, the proof of plaintext equality under two public keys used by every rotation, and the proofs Π_{Lin} and Π_{Bnd} of every distributed decryption. By the adaptive reprogramming bound of [89] the total loss is $O((n_B + \Theta)q_H \sqrt{q_H} |\mathcal{R}|^{-1/2})$, which is negligible.

Game C_2 . Every value opened during the cleansing is replaced by a uniform element of the slot space. A masked opening reveals $x + \sum_j \gamma_j$ where the sum runs over the trustees of the quorum. At least one trustee is honest because at most $t - 1$ of them are corrupted, and that trustee draws γ_j uniformly and independently of everything else, so the revealed value is uniform and independent of x . The same holds for the rotations, whose masks are drawn afresh. Games C_1 and C_2 are therefore identically distributed.

Game C_3 . Every distributed decryption transcript is replaced by the output of $S_{\Pi_{\text{Dec}}}$ on the value that the combination must produce, which is the uniform value fixed in C_2 for an intermediate opening

and the cleansed vote list for the final one. As in Section A.4 the statistical distance is at most $2^{-\text{sec}}$ per ciphertext and per trustee, so the two games are within $\Theta t 2^{-\text{sec}}$.

Game C_4 . Every ciphertext produced by an honest party is replaced by an encryption of zero. The ciphertexts concerned are the roster entries, the two ciphertexts of each ballot cast by an honest voter, the encryptions of the masks and the re-encryptions performed inside the openings. After C_2 and C_3 no plaintext of those ciphertexts is used anywhere in the transcript, since every opened value is uniform and every decryption transcript is simulated, so a hybrid over the ciphertexts gives a loss of $(n_V + 2n_B + O(\Theta))\varepsilon_{\text{MLWE}}$.

Game C_5 . The output of the mix-net is replaced by a uniformly random permutation of the multiset of cleansed votes, and the shuffle proofs are replaced by simulated ones. At least one mix server is honest and draws its permutation uniformly, so the composition is uniform; the proofs were already simulated in C_1 . This game is identically distributed to C_4 .

The view is a function of the ideal data. In C_5 the transcript seen by $\mathcal{A}$ consists of the public parameters, of the roster, which is a list of n_V encryptions of zero, of the board, which is a list of pairs of encryptions of zero together with simulated proofs and of the ballots that $\mathcal{A}$ itself posted, of a sequence of Θ uniform slot vectors with simulated decryption transcripts, of a uniformly permuted list of the cleansed votes, and of the announced result. Only two quantities of the real election influence that view, which are the number of entries posted to the board and the multiset of counted votes. The first is $|D|$ in the ideal experiment and the second is $\text{Result}(\text{Cleanse}(D))$. The disclosed credential cred is a uniform string of length λ in both cases, because FakeCred samples uniformly and because a real credential is sampled uniformly by the registrar, and after C_4 it appears nowhere else in the transcript. The adversary therefore learns nothing that separates $\beta = 0$ from $\beta = 1$ beyond what the ideal experiment reveals; in particular the cleansing does not disclose how many entries share a credential, which is the leak that the scheme of [17] exhibits and that [20] removes.

The ideal adversary. Define $\mathcal{B}^*$ to run $\mathcal{A}$ on a transcript built as in C_5 from the data that the ideal experiment provides. It first receives $\mathcal{D}$ and (j, α) from $\mathcal{A}$ and forwards them, then receives $|D|$ and produces a board of that length filled with encryptions of zero and simulated proofs, choosing for the entries of the corrupted voters the ballots that $\mathcal{A}$ posts and reporting the corresponding v_i to the ideal experiment, then receives r^* and produces the permuted vote list and the simulated cleansing and decryption transcripts consistent with r^* , and finally outputs the bit that $\mathcal{A}$ outputs. By construction the view of $\mathcal{A}$ inside $\mathcal{B}^*$ is the view of C_5 , so

$$\left| \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{CR-Real}} = 1] - \Pr[\text{Exp}_{\Pi, \mathcal{B}^*}^{\text{CR-Ideal}} = 1] \right| \leq \Theta t 2^{-\text{sec}} + (n_V + 2n_B + O(\Theta))\varepsilon_{\text{MLWE}} + \varepsilon_{\text{zk}},$$

which is negligible in λ . $\square$

A.7 Proof of Theorem 8.2

The verification algorithm of Oylama performs four kinds of checks, and we treat them in turn.

First, it checks the ballots. As in Section A.5, Valid accepts an entry only when its two proofs of knowledge verify and its digest is new,

so by the soundness of those proofs in the quantum random oracle model [91] every accepted entry has a well defined pair of plaintexts, which are an option and a credential, and by the collision resistance of SHAKE256 no entry is counted twice.

Second, it checks the packed cleansing. Every step of the cleansing is one of three operations. An addition or a multiplication by a public plaintext is a deterministic function of published ciphertexts, so the verifier recomputes it. An application of the automorphism $X \mapsto X^{3^t}$ is likewise deterministic. A masked opening publishes the encryptions of the masks, the combined ciphertext, the decryption transcript and the recomposed result; the verifier recomputes the combined ciphertext from the published masks, checks the decryption transcript as in Claim 2 of Section A.5, checks the proof that each trustee multiplied the opened value by the ciphertext it committed to, and checks the proof that the two encryptions of a mask under the two public keys carry the same plaintext. Since the commitment scheme is binding under MSIS and each of these proofs is sound, a transcript that verifies determines the same sequence of plaintexts as an honest evaluation, except with probability $O(\Theta)(\epsilon_{\text{MSIS}} + |C|^{-1} + q_{\text{H}}^2 \epsilon_{\text{ss}})$.

Third, it checks the mix-net and the final decryption, which is Claims 1 and 2 of Section A.5 applied to the packed parameters.

Fourth, it recomputes the result from the decrypted options. Combining the four items, a verifying transcript proves that the announced result is the image under Result of the multiset obtained by applying the cleansing circuit to the accepted entries. That circuit keeps, for each credential of the roster, the accepted entry with the largest board position among those carrying that credential, and replaces the option of every other entry by the invalid option. Hence a checked honest voter, whose last entry is on the board by the same argument as in Section A.5 and whose credential is in the roster, contributes its own option; an honest voter that does not check contributes its option or nothing; and the corrupted voters contribute at most one option each, so at most $|CU|$ options in total. This is the decomposition required by Definition 4.2, so both experiments return 0 except with the probability displayed above. $\square$

References

- [1] B. Adida. Helios: Web-based open-audit voting. In *USENIX Security*, pages 335–348, 2008.
- [2] V. Cortier, P. Gaudry, and S. Glondu. Belenios: A simple private and verifiable electronic voting system. In *Foundations of Security, Protocols, and Equational Reasoning*, volume 11565 of *LNCS*, Springer, 2019.
- [3] V. Cortier, D. Galindo, S. Glondu, and M. Izabachène. Election verifiability for Helios under weaker trust assumptions. In *ESORICS*, pages 327–344, Springer, 2014.
- [4] J. Benaloh et al. ElectionGuard: A cryptographic toolkit to enable verifiable elections. In *USENIX Security*, pages 5485–5502, 2024.
- [5] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219, ACM, 1996.
- [6] I. Chillotti et al. A homomorphic LWE based e-voting scheme. In *PQCrypto*, pages 245–265, Springer, 2016.
- [7] R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
- [8] X. Boyen, T. Haines, and J. Müller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. In *IEEE EuroS&P*, pages 272–291, 2021.
- [9] P. Hough, C. Sandsbråten, and T. Silde. More efficient lattice-based electronic voting from NTRU. *IACR Commun. Cryptol.*, 1(4), 2025.
- [10] O. Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), 2009.
- [11] Vadim Lyubashevsky. Fewer keys and more efficiency from the ring-lwe cryptosystem. In *CRYPTO*, volume 7417 of *LNCS*, pages 172–190, Springer, Berlin, Heidelberg, 2012. Presented at EUROCRYPT 2012, but formally introduces the module setting and is the basis for its adoption in subsequent schemes.
- [12] A. Langlois and D. Stehle. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.*, 75(3):565–599, 2015.
- [13] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 41–69, Springer, 2011.
- [14] Diego F. Aranha, Carsten Baum, Kristian Gjøsteen, and Tjerand Silde. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*, page 1467–1481, New York, NY, USA, 2023. Association for Computing Machinery.
- [15] V. Cortier, J. Lallemand, and B. Warinschi. Fifty shades of ballot privacy: Privacy against a malicious board. In *IEEE CSF*, pages 17–32, 2020.
- [16] C.C. Drăgan et al. Machine-checked proofs of privacy against malicious boards for Selene & co. In *IEEE CSF*, pages 335–347, 2022.
- [17] Ari Juels, Dario Catalano, and Markus Jakobsson. Coercion-resistant electronic elections. In *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society (WPES)*, pages 61–70, ACM, 2005.
- [18] Michael R. Clarkson, Stephen Chong, and Andrew C. Myers. Civitas: Toward a secure voting system. In *2008 IEEE Symposium on Security and Privacy (SP 2008)*, pages 354–368, IEEE, 2008.
- [19] Wouter Lueks, Iñigo Querejeta-Azurmendy, and Carmela Troncoso. Voteagain: a scalable coercion-resistant voting system. In *Proceedings of the 29th USENIX Conference on Security Symposium, SEC’20*, USA, 2020. USENIX Association.
- [20] Véronique Cortier, Pierrick Gaudry, and Quentin Yang. Is the jcj voting system really coercion-resistant? In *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*, pages 186–200, 2024.
- [21] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, NIST, 2024.
- [22] Y. Wang et al. Threshold key encapsulation mechanisms from module lattices. In *Information Security and Privacy (ACISP)*, pages 289–308, Springer, 2025.
- [23] National Institute of Standards and Technology. Threshold schemes for module-lattice key encapsulation. NIST Threshold Cryptography Workshop (TCall), 2025.
- [24] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report FIPS 204, NIST, 2024.
- [25] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS 202, NIST, 2015.
- [26] National Institute of Standards and Technology. Advanced encryption standard (AES). Technical Report FIPS 197, NIST, 2001.
- [27] M. Dworkin. Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC. Technical Report SP 800-38D, NIST, 2007.
- [28] D.F. Aranha, C. Baum, K. Gjøsteen, and T. Silde. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. *Cryptology ePrint Archive*, Paper 2022/422, 2022.
- [29] X. Boyen, T. Haines, and J. Müller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. *Cryptology ePrint Archive*, Paper 2021/304, 2021.
- [30] R. Küsters, T. Truderung, and A. Vogt. Verifiability, privacy, and coercion-resistance: New insights from a case study. In *IEEE S&P*, pages 538–553, 2011.
- [31] D. Bernhard, V. Cortier, D. Galindo, O. Pereira, and B. Warinschi. Sok: A comprehensive analysis of game-based ballot privacy definitions. In *IEEE S&P*, pages 499–516, 2015.
- [32] J. Benaloh. Verifiable secret-ballot elections. In *Ph.D. Thesis, Yale University*, 1987.
- [33] R. Küsters, T. Truderung, and A. Vogt. Accountability: Definition and relationship to verifiability. In *ACM CCS*, pages 526–535, 2010.
- [34] V. Cortier, D. Galindo, R. Küsters, J. Müller, and T. Truderung. Sok: Verifiability notions for e-voting protocols. In *IEEE S&P*, pages 779–798, 2016.
- [35] D.F. Aranha, C. Baum, K. Gjøsteen, and T. Silde. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *ACM CCS*, pages 1467–1481, 2023.
- [36] G. Kaim et al. Post-quantum online voting scheme. In *FC Workshops*, pages 290–305, Springer, 2021.
- [37] S. Bouaziz-Ermann et al. Lattice-based (partially) blind signature without restart. *Cryptology ePrint Archive*, Report 2020/260, 2020.
- [38] D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *LNCS*, pages 700–718, Springer, 2012.
- [39] R. Bendlin, S. Krehbiel, and C. Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In *ACNS*, volume 7954 of *LNCS*, pages 218–236, Springer, 2013.
- [40] C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *ACM STOC*, pages 197–206, 2008.
- [41] V. Lyubashevsky, C. Peikert, and O. Regev. A toolkit for ring-LWE cryptography. In *EUROCRYPT*, volume 7881 of *LNCS*, pages 35–54, Springer, 2013.
- [42] E. Hauck et al. Lattice-based blind signatures, revisited. In *CRYPTO*, volume 12171 of *LNCS*, Springer, 2020.
- [43] R. Mosaheb. *Quantum-Safe Electronic Voting Schemes*. PhD thesis, Université du Luxembourg, 2025.
- [44] A. Esser, A. May, and F. Zwegdinger. McEliece needs a break – solving McEliece-1284 and quasi-cyclic-2918 with modern ISD. In *EUROCRYPT*, volume 13277 of *LNCS*, pages 433–457, Springer, 2022.
- [45] I. Black et al. Practical quantum-safe voting from lattices, extended. *Cryptology ePrint Archive*, Paper 2022/1686, 2022.
- [46] Chen Wu, Shaohua Tang, and Xingfu Yan. A homomorphic lwe-based verifiable electronic voting system. In *2018 IEEE Conference on Dependable and Secure*

- Computing (DSC)*, pages 1–8, 2018.
- [47] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In *Cryptographers Track at the RSA Conference*, pages 319–339. Springer, 2011.
- [48] M.R. Albrecht, R. Player, and S. Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.
- [49] N. Abapour et al. Blockchain-based economic voting with posterior security from lattices. In *IWSEC*, pages 247–266. Springer, 2025.
- [50] I. Hwang, J. Seo, and Y. Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *CRYPTO*, pages 414–448. Springer, 2024.
- [51] T.H. Yuen et al. Posterior security: Anonymity and message hiding of standard signatures. In *ACM CCS*, pages 528–542, 2025.
- [52] F. Baldimtsi et al. Non-interactive blind signatures: Post-quantum and stronger security. In *ASIACRYPT*, volume 15485 of *LNCS*. Springer, 2025.
- [53] S. Park et al. Going from bad to worse: From internet voting to blockchain voting. *J. Cybersecur.*, 7(1), 2021.
- [54] P. Sekar et al. Quantum-resistant electronic voting system using lattice cryptography techniques. In *Internet and Modern Society*, pages 257–270. Springer, 2025.
- [55] J. Bos et al. CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. *Cryptology ePrint Archive*, Report 2017/634, 2017.
- [56] J. Huang et al. The application of the blockchain technology in voting systems. *ACM Comput. Surv.*, 54:1–28, 2021.
- [57] S. Vivek et al. E-voting systems using blockchain: An exploratory literature survey. In *IEEE ICIRCA*, 2020.
- [58] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, NIST, 2015.
- [59] S.P. Mahanayak, B. Nikhita, and S. Bilgaiyan. Enhancing e-voting security with quantum-resistant encryption. *SN Comput. Sci.*, 4(5), 2023.
- [60] W. Diffie and M.E. Hellman. New directions in cryptography. *IEEE Trans. Inf. Theory*, 22(6):644–654, 1976.
- [61] B. Guidi, A. Michienzi, and L. Ricci. Data persistence in decentralized social applications: the IPFS approach. In *IEEE CCNC*, 2021.
- [62] Y. Lin and C. Zhang. A method for protecting private data in IPFS. In *IEEE CSCWD*, 2021.
- [63] A. Junior Gabriel et al. Post-quantum cryptography system for secure electronic voting. *Open Comput. Sci.*, 9(1):292–298, 2019.
- [64] A.J. Gabriel et al. Post-quantum cryptography: A combination of post-quantum cryptography and steganography. In *ICITST*, pages 454–457, 2013.
- [65] A. Petzoldt. *Selecting and Reducing Key Sizes for Multivariate Cryptography*. PhD thesis, Technical University Darmstadt, 2013.
- [66] J. P. Cardoso Barbosa, G. Zamboni, T. Bardini Idalino, and R. Custódio. From ElGamal to lattice-based: Enhancing the Helios voting system with quantum-safe cryptography. In *Applied Cryptography and Network Security (ACNS)*, pages 29–49. Springer, 2025.
- [67] Diego F. Aranha, Carsten Baum, Kristian Gjøsteen, Tjerand Silde, and Thor Tunge. Lattice-based proof of shuffle and applications to electronic voting. In *Topics in Cryptology – CT-RSA 2021*, volume 12704 of *Lecture Notes in Computer Science*, pages 227–251. Springer, 2021.
- [68] Carsten Baum, Ivan Damgård, Vadim Lyubashevsky, Sabine Oechsner, and Chris Peikert. More efficient commitments from structured lattice assumptions. In *Security and Cryptography for Networks – SCN 2018*, volume 11035 of *Lecture Notes in Computer Science*, pages 368–385. Springer, 2018.
- [69] S. Agrawal, D. Boneh, and X. Boyen. Efficient lattice (H)IBE in the standard model. In *EUROCRYPT*, pages 553–572, 2010.
- [70] R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
- [71] R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
- [72] D. Stehlé and R. Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In *EUROCRYPT*, volume 6632 of *LNCS*, pages 27–47. Springer, 2011.
- [73] C. Baum et al. More efficient commitments from structured lattice assumptions. In *SCN*, volume 11035 of *LNCS*, pages 368–385. Springer, 2018.
- [74] D.F. Aranha et al. Lattice-based proof of shuffle and applications to electronic voting. In *CT-RSA*, volume 12704 of *LNCS*, pages 227–251. Springer, 2021.
- [75] D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *ACM CCS*, pages 1467–1481, 2023.
- [76] J. Bootle et al. More efficient amortization of exact zero-knowledge proofs for LWE. In *ESORICS*, volume 12973 of *LNCS*, pages 608–627. Springer, 2021.
- [77] N. Aragon et al. BIKE: Bit-flipping key encapsulation. https://bikesuite.org/files/v5.0/BIKE_Spec.2022.10.10.1.pdf, 2022.
- [78] C. Aguilar-Melchor et al. Efficient encryption from random quasi-cyclic codes. *IEEE Trans. Inf. Theory*, 64(5):3927–3943, 2018.
- [79] E. Bellini et al. Enhancing code based zero-knowledge proofs using rank metric. In *CANS*, volume 12579, pages 570–592, 2020.
- [80] F. Benhamouda et al. Better zero-knowledge proofs for lattice encryption and their application to group signatures. In *ASIACRYPT*, pages 551–572, 2014.
- [81] V. Lyubashevsky. Lattice signatures without trapdoors. In *EUROCRYPT*, pages 738–755, 2012.
- [82] R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
- [83] L.ucas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, volume 9056 of *LNCS*, pages 617–640. Springer, 2015.
- [84] L.ucas et al. Lattice signatures and bimodal gaussians. In *CRYPTO*, volume 8042 of *LNCS*, pages 40–56. Springer, 2013.
- [85] F. Laguillaumie et al. Lattice-based group signatures with logarithmic signature size. In *ASIACRYPT*, volume 8270 of *LNCS*, pages 41–61. Springer, 2013.
- [86] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. Homomorphic encryption standard. Technical report, HomomorphicEncryption.org, 2018.
- [87] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) Fully Homomorphic Encryption without Bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS)*, pages 309–325. ACM, 2012.
- [88] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In *Advances in Cryptology – CRYPTO 2022*, volume 13508 of *LNCS*, pages 71–101. Springer, 2022.
- [89] A. B. Grilo, K. Hövelmanns, A. Hülsing, and C. Majenz. Tight adaptive reprogramming in the QROM. In *Advances in Cryptology – ASIACRYPT 2021*, volume 13090 of *LNCS*, pages 637–667. Springer, 2021.
- [90] E. Kiltz, V. Lyubashevsky, and C. Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In *Advances in Cryptology – EUROCRYPT 2018*, volume 10822 of *LNCS*, pages 552–586. Springer, 2018.
- [91] J. Don, S. Fehr, C. Majenz, and C. Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In *Advances in Cryptology – CRYPTO 2019*, volume 11693 of *LNCS*, pages 356–383. Springer, 2019.